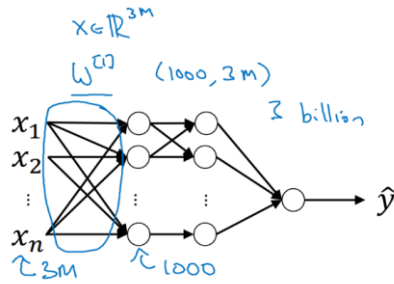


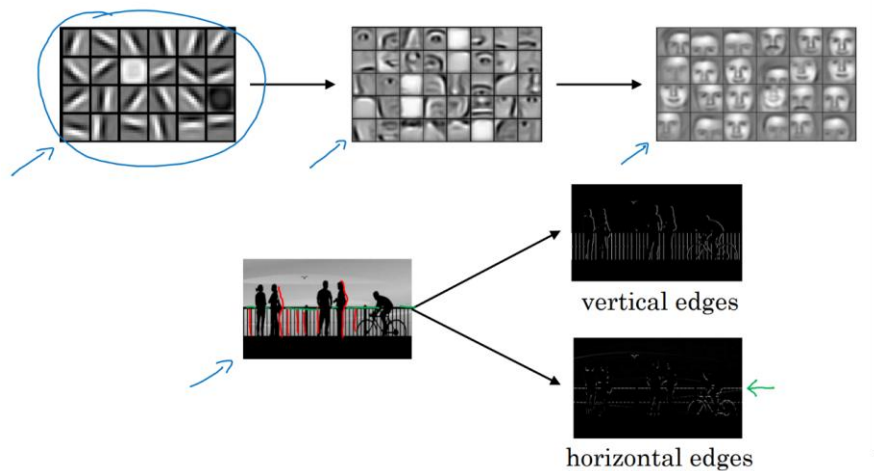
Computer Vision

- Applications: image classification, object detection, neural style transfer
- Deep learning on large images



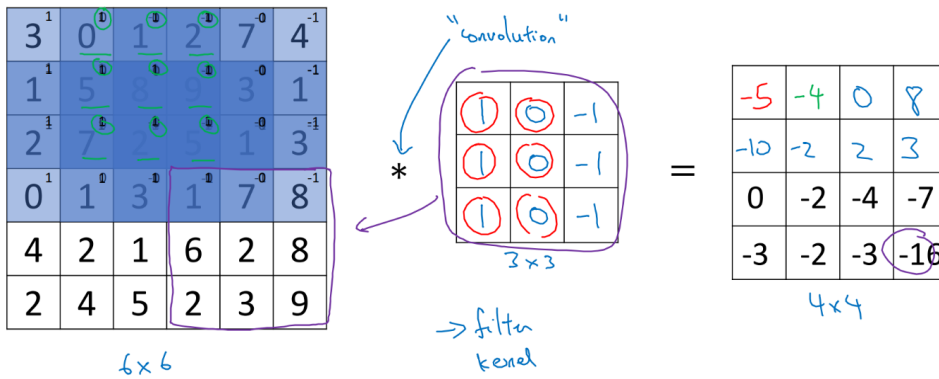
Edge Detection – Convolution Example

- Convolution is a fundamental building block of ML
- Early layers detect edges, middle layers detect objects, later layers detect faces

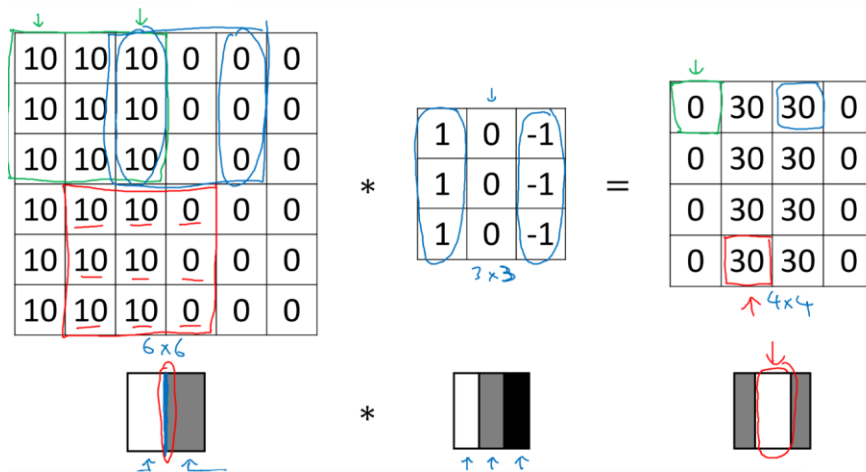


- Vertical edge detection
 - Convolve (*) the input matrix by a **filter/kernel** [1 bright pixels, 0, -1 dark pixels]
 - Steps: paste the filter over the top left corner of the original input, take element-wise product. Adding up these numbers gives the corresponding value of the resulting matrix. Then shift the paste left by one column and repeat for the rest of the row. Then shift the whole block down one row, repeat, etc.
 - In base Python `convolve = conv-forward`
 - Tensorflow: `tf.nn.conv2d`
 - Keras: `Conv2D`

$$\rightarrow 3 \times 1 + 1 \times 1 + 2 \times 1 + 0 \times 0 + 5 \times 0 + 7 \times 0 + 1 \times 1 + 8 \times 1 + 2 \times 1 = -5$$



- Example below has a strong vertical edge right down the middle
- Convoluting using the same 3x3 results in 30s down the middle: the vertical edge was detected



- Dark-to-light vs light-to-dark transitions
 - 10s on left → 30 down middle → light-to-dark transition
 - 10s on right → -30 down middle → dark-to-light transition
 - If you don't care about the direction, you can take the absolute value

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0

→

1	0	-1
1	0	-1
1	0	-1

*

0	30	30	0
0	30	30	0
0	30	30	0
0	30	30	0

=

0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

→

1	0	-1
1	0	-1
1	0	-1

*

0	-30	-30	0
0	-30	-30	0
0	-30	-30	0
0	-30	-30	0

=

- Vertical and horizontal edge detection
 - Vertical: Pixels bright on left and dark on right
 - Horizontal: Pixels bright on top and dark on bottom

1	0	-1
1	0	-1
1	0	-1

Vertical

1	1	1
0	0	0
-1	-1	-1

Horizontal

10	10	10	0	0	0
10	10	10	0	0	0
10	10	10	0	0	0
0	0	0	10	10	10
0	0	0	10	10	10
0	0	0	10	10	10

6x6

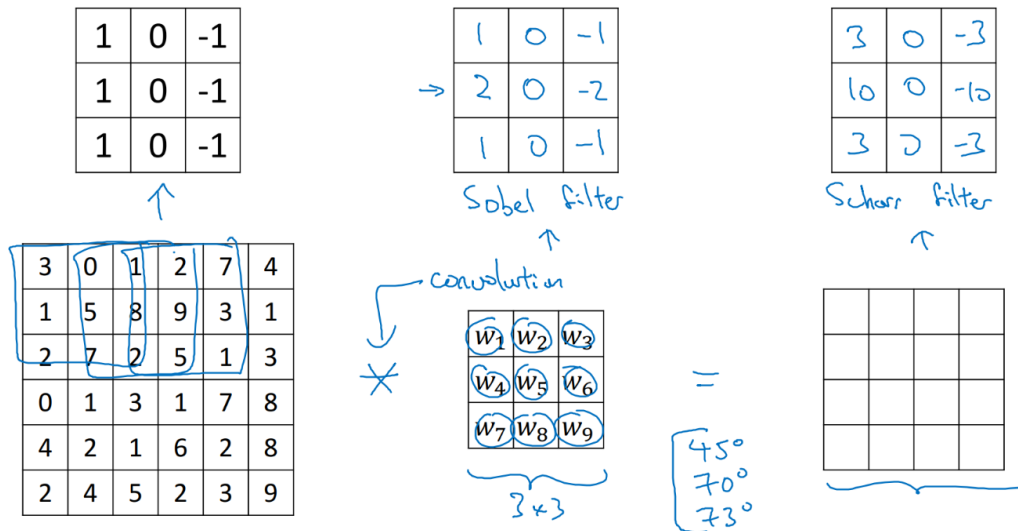
*

1	1	1
0	0	0
-1	-1	-1

=

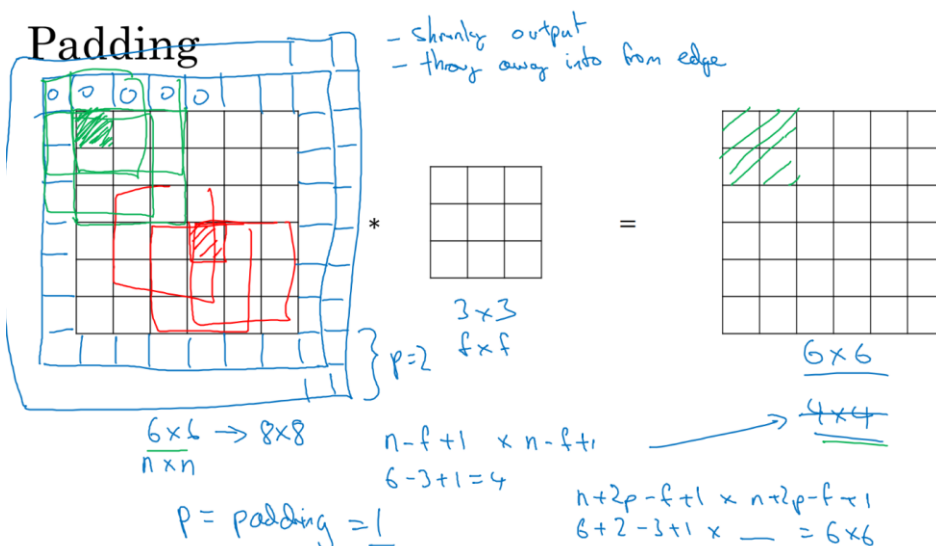
0	0	0	0
30	10	-10	-30
30	10	-10	-30
0	0	0	0

- Learning to detect edges
 - You can use many different filters with different properties (e.g. Sobel, Scharr)
 - Don't have to pick these 9 numbers, you can learn them and treat the 9 numbers of the matrix as parameters that you can learn via backpropagation
 - NN can learn features even more robustly



Padding

- Adding additional border to avoid shrinking the output
- If you have an $n \times n$ image and convolute with an $f \times f$ filter, then the dimension of the output will be $n-f+1 \times n-f+1$
- When padded, the output is $n+2p-f+1 \times n+2p-f+1$



- Two common types of padding for convolutions

- Valid: no padding

- $p=0$

$$\begin{array}{ccc} n \times n & * & f \times f \rightarrow \frac{n-f+1}{1} \times \frac{n-f+1}{1} \\ 6 \times 6 & * & 3 \times 3 \rightarrow 4 \times 4 \end{array}$$

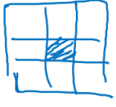
- Same: pad so that output size is the same as the input size

- $p = (f-1)/2$

- f is usually odd (keeps p even/ adding symmetric, and has a central position to use as a reference for positioning)

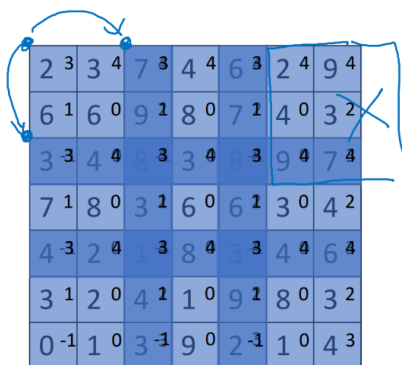
$$\begin{array}{l} n+2p-f+1 \times n+2p-f+1 \\ n+2p-f+1 = n \Rightarrow p = \frac{f-1}{2} \\ 3 \times 3 \quad p = \frac{3-1}{2} = 1 \quad \left| \begin{array}{c} 5 \times 5 \\ f=5 \end{array} \right. \quad p=2 \end{array}$$

f is usually odd



Strided Convolutions

- Moving the box over/down by s steps (instead of default 1 step)
- Summary of convolutions: $n \times n$ image, $f \times f$ filter, padding p , stride s , output size $(n+2p-f)/s + 1 \times (n+2p-f)/s + 1$ (round down if not an integer)
- If any of the box doesn't fit in the matrix, do not include it



$$\begin{array}{ccc} \begin{array}{|c|c|c|} \hline 3 & 4 & 4 \\ \hline 1 & 0 & 2 \\ \hline -1 & 0 & 3 \\ \hline \end{array} & = & \begin{array}{|c|c|c|} \hline 91 & 100 & 83 \\ \hline 69 & 91 & 127 \\ \hline 44 & 72 & 74 \\ \hline \end{array} \\ \text{3x3} & & \text{3x3} \\ \text{stride } s=2 & & \lfloor z \rfloor = \text{floor}(z) \end{array}$$

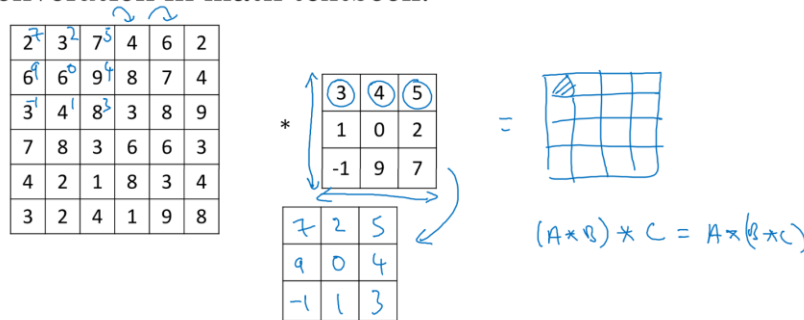
$$\begin{array}{l} n \times n \quad * \quad f \times f \\ \text{padding } p \quad \text{stride } s \\ s=2 \end{array}$$

$$\left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n+2p-f}{s} + 1 \right\rfloor$$

$$\frac{7+0-3}{2} + 1 = \frac{4}{2} + 1 = 3$$

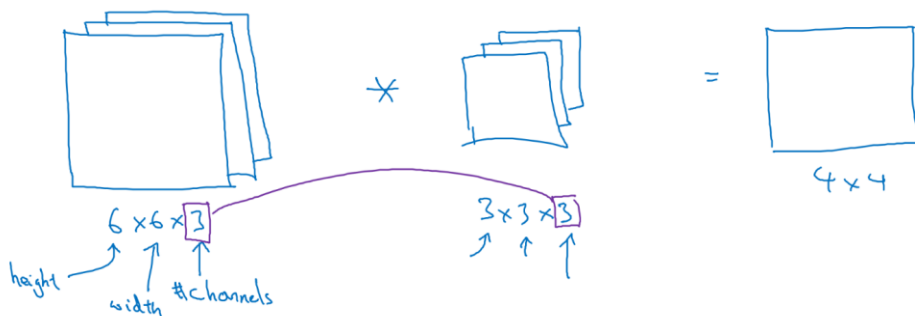
- Technical note on cross-correlation vs convolution
 - Most books transpose the filter matrix before convolving
 - This gives the associativity property $(A * B) * C = A * (B * C)$
 - Not transposing (as done above) is called **cross-correlation**, but by convention in ML they just call it convolution

Convolution in math textbook:

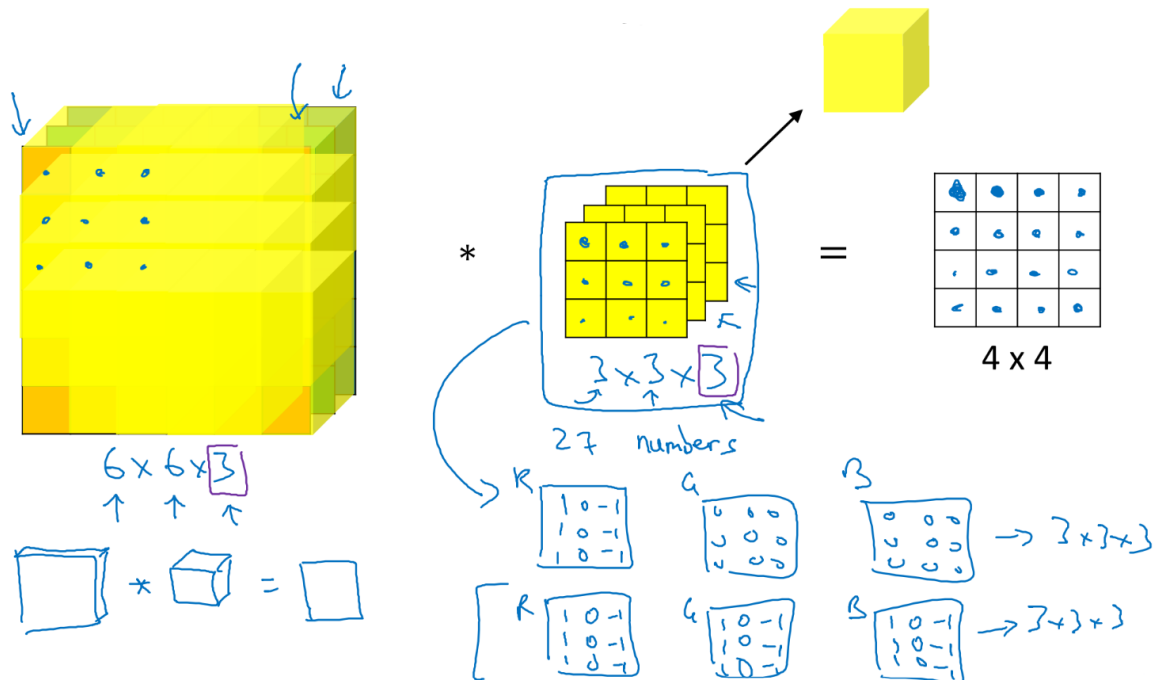


Convolutions Over Volumes

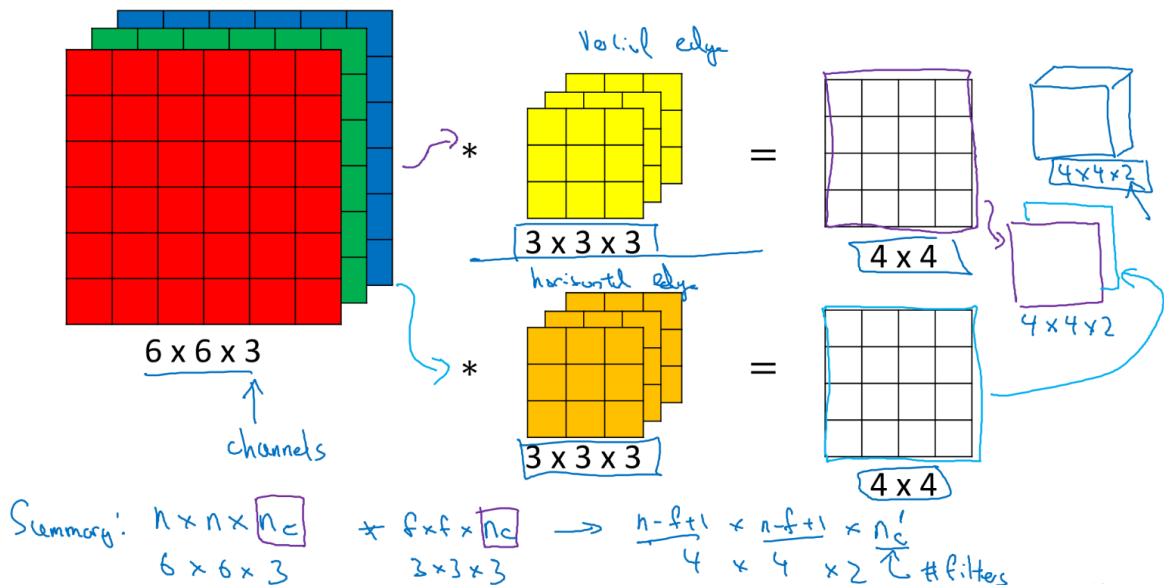
- Convolve with a 3D filter ($3 \times 3 \times 3$)
- Volume dimension/depth (# channels) of input matrix and 3D filter have to be the same
- Output matrix is still flat/volume dimension is 1
- Convolution on RGB images



- 27 filter parameters to multiply by corresponding 27 numbers in the input
- If you want to detect edges in red channel, R filter is $[1 \ 0 \ -1]$, G B filters are $[0 \ 0 \ 0]$
- Edge detector in any color, R C G filters all $[1 \ 0 \ -1]$

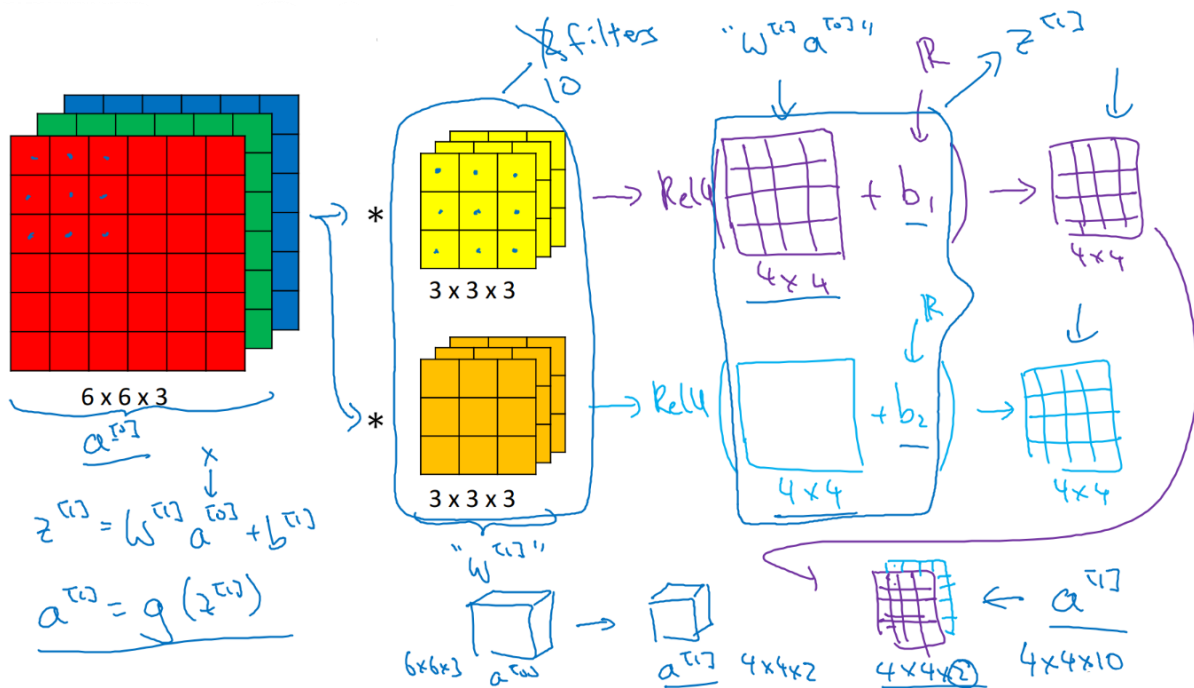


- What if you want to detect vertical edges and horizontal edges (an/or others)?
- Multiple filters
 - Convolve with each filter separately, then stack the filter outputs into a 3D result
 - $n \times n \times n_c * f \times f \times f_c \rightarrow (n-f+1) \times (n-f+1) \times n'_c$ ($n'_c = \# \text{ filters}$; no padding, stride 1)

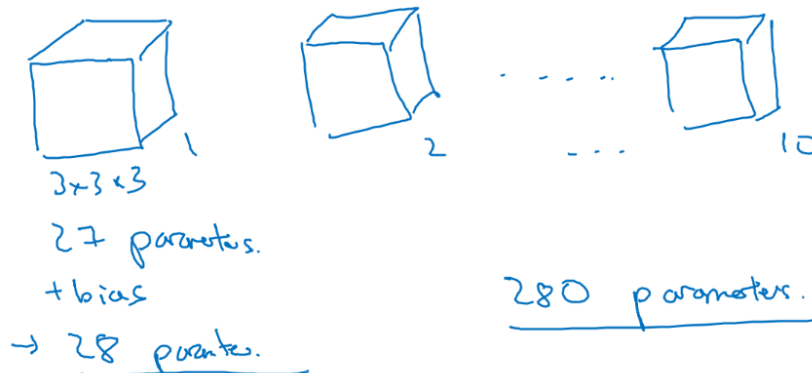


One Layer of a Convolutional Network

- Example
 - Filters play a similar role to $W^{[1]}$, and the input is $a^{[0]}$. Add a bias (role of $z^{[1]}$) and apply non-linearity to each convolution output ($W^{[1]}a^{[0]}$), then stack up ($a^{[1]}$)



- Number of parameters in one layer
 - If you have 10 filters that are $3 \times 3 \times 3$ in one layer of a neural network, how many parameters does that layer have?
 - $(f \times f \times f_c + 1) * n_c$
 - Once you've learned 10 feature detectors that work, you could apply this even to large images, and the number of parameters is still fixed and relatively small



- Summary of notation

If layer l is a convolution layer:

$f^{[l]}$ = filter size
 $p^{[l]}$ = padding
 $s^{[l]}$ = stride
 $n_c^{[l]}$ = number of filters

→ Each filter is: $f^{[l-1]} \times f^{[l-1]} \times n_c^{[l-1]}$
 Activations: $a^{[l-1]} \rightarrow n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$
 Weights: $f^{[l-1]} \times f^{[l-1]} \times n_c^{[l-1]} \times n_c^{[l]}$
 bias: $n_c^{[l]} - (1, 1, 1, n_c^{[l]})$ ← #filters in layer l .

Input: $n_H^{[l-1]} \times n_W^{[l-1]} \times n_c^{[l-1]}$
 Output: $n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$

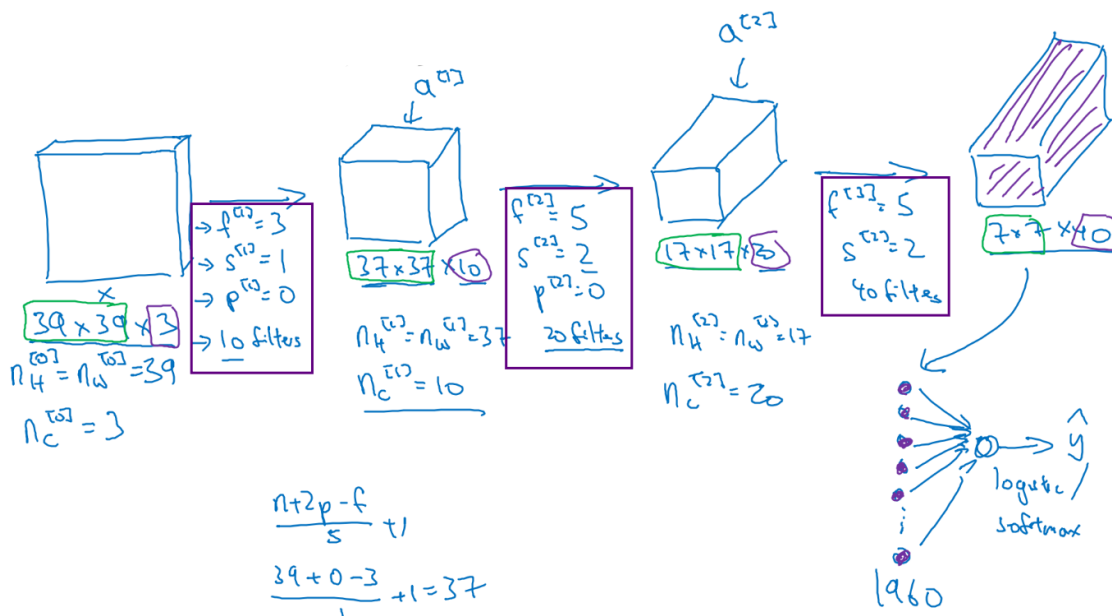
$$n_{HW}^{[l]} = \left\lfloor \frac{n_H^{[l-1]} + 2p^{[l]} - f^{[l]}}{s^{[l]}} + 1 \right\rfloor$$

$$A^{[l]} \rightarrow m \times n_H^{[l]} \times n_W^{[l]} \times n_c^{[l]}$$

$$n_c^{[l]} - (1, 1, 1, n_c^{[l]})$$

Simple ConvNet Example

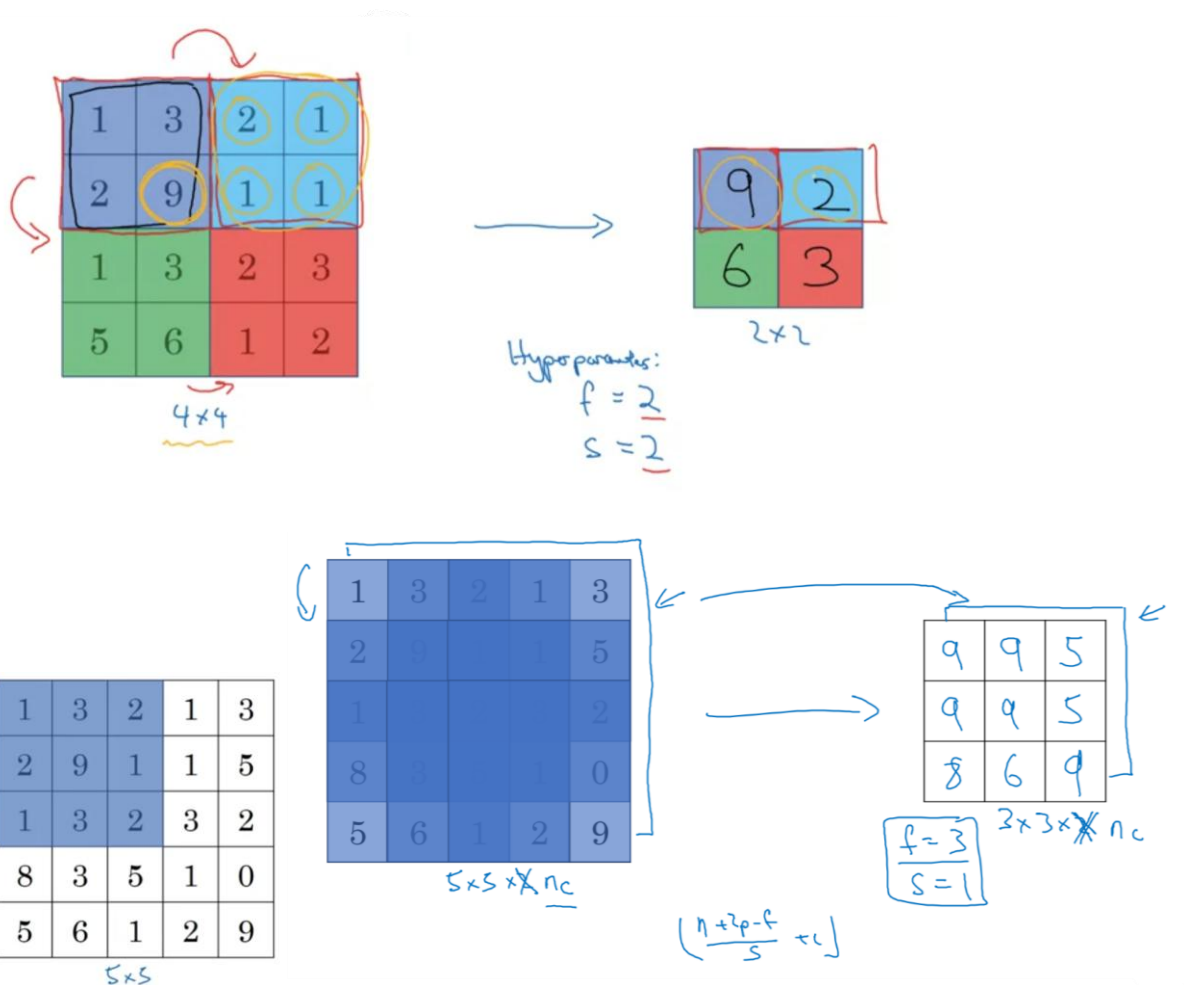
- $p=0 \rightarrow$ valid convolution
- Flatten out final output stack into a vector and feed into logistic or softmax unit to make prediction for final output
- Typically you start with larger images, height and width will stay the same for a while then gradually trend down as you go deeper in the NN, whereas the number of channels will generally increase



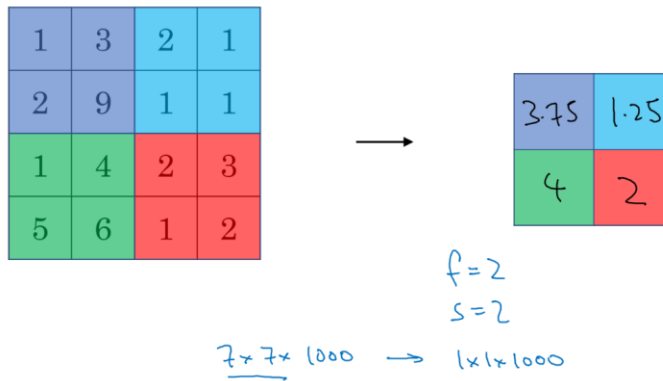
- Types of layer in a convolutional network: Convolution, Pooling, Fully connected (FC)

Pooling Layers

- Reduce the size of the representation to speed up computation as well as make some of the features it detects a bit more robust
- Max pooling
 - Each cell of the output = max from the corresponding shaded region of the input
 - If features are detected anywhere in the quadrant, it remains preserved in the output of max pooling
 - No parameters to learn, no gradient descent, just fixed computation with f and s
 - If 3D input, perform computation on each channel independently



- Average pooling
 - Isn't used very often
 - Very deep in a NN, you might use average pooling to collapse your representation
 - Each cell of the output = average of each corresponding shaded region

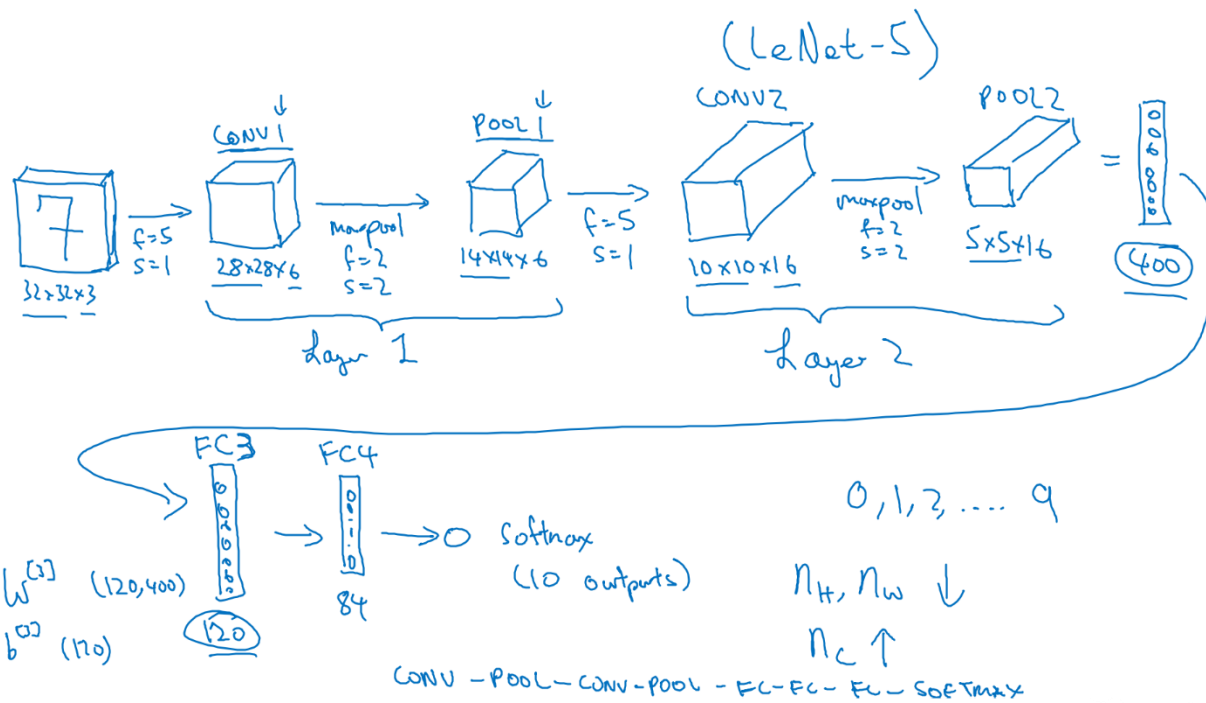


- Summary of pooling
 - Hyperparameters:
 - f = filter size (commonly $f=2$)
 - s = stride (commonly $s=2$)
 - Max or average pooling
 - Can add padding (very rarely used)
 - No parameters to learn!

$$\begin{aligned}
 & n_H \times n_W \times n_C \\
 & \downarrow \\
 & \left\lfloor \frac{n_H - f}{s} + 1 \right\rfloor \times \left\lfloor \frac{n_W - f}{s} + 1 \right\rfloor \\
 & \times n_C
 \end{aligned}$$

ConvNet Example with Gully Connected Layers

- Example inspired by LeNet5 created by Yann LeCun
- When counting layers, people often just count the number of layers that have weights/parameters, so pooling layers can be grouped in (e.g. Conv 1 and Pool 1)
- Image dimensions shrink over time, # channels increases, flatten output into vector of and feed into a fully connected layer (each of the 400 units from the flattened vector is connected to each of the 120 units of the FC layer), feed to another FC layer, then feed into softmax output layer
- Common to have a few Conv $\rightarrow$ Pool and at the end a few FC $\rightarrow$ FC and output softmax

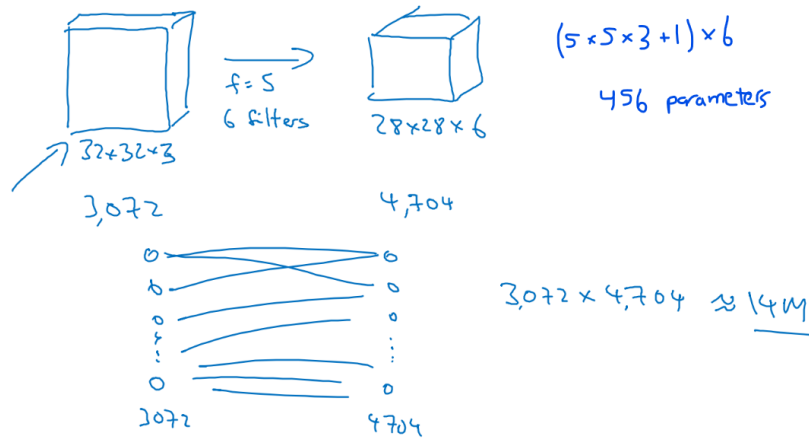


- Pool layers have no parameters, conv layers have few parameters, most parameters are in FC layers, activation size generally decreases throughout

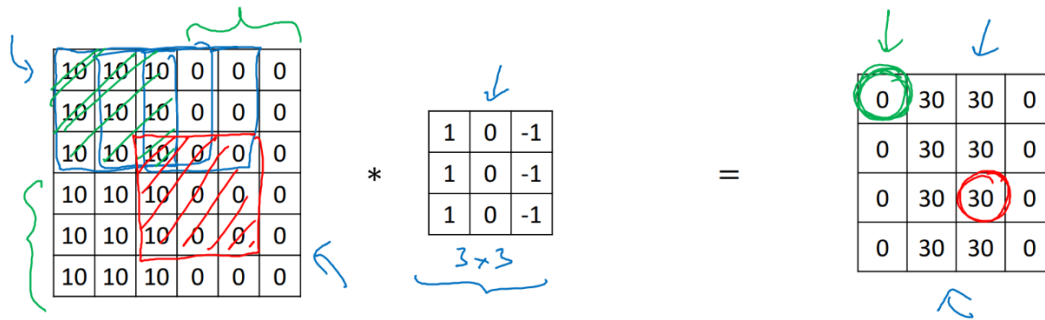
	Activation shape	Activation Size	# parameters
Input:	(32,32,3)	3,072 $a^{(0)}$	0
CONV1 ($f=5, s=1$)	(28,28,6)	4,704	456 ←
POOL1	(14,14,6)	1,176	0 ←
CONV2 ($f=5, s=1$)	(10,10,16)	1,600	2,416 ←
POOL2	(5,5,16)	400	0 ←
FC3	(120,1)	120	48,120 }
FC4	(84,1)	84	10,164 }
Softmax	(10,1)	10	850

Why Convolutions?

- Vastly reduces the number of parameters to train

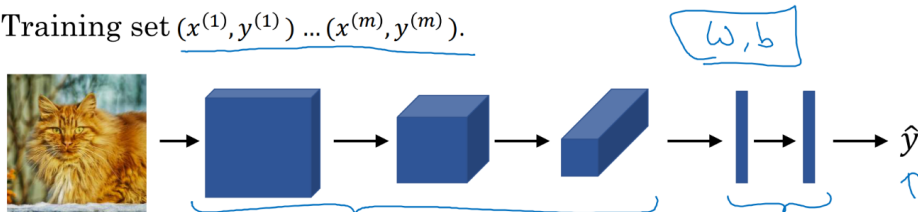


- Advantages (how ConvNets achieve so few parameters)
 - **Parameter sharing:** a feature detector (such as a vertical edge detector) that's useful in one part of the image is probably useful in another part of the image
 - **Sparsity of connections:** in each layer, each output value depends only on a small number of inputs
 - Also, good at capturing **translation invariance:** robust to shifting the image



- Putting it together

Training set $(x^{(1)}, y^{(1)}) \dots (x^{(m)}, y^{(m)})$.



$$\text{Cost } J = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$$

Use gradient descent to optimize parameters to reduce J

What do you think applying this filter to a grayscale image will do?

$$\begin{bmatrix} 0 & 1 & -1 & 0 \\ 1 & 3 & -3 & -1 \\ 1 & 3 & -3 & -1 \\ 0 & 1 & -1 & 0 \end{bmatrix}$$

Detect horizontal edges

Detect image contrast

Detect vertical edges

Detect 45 degree edges

Suppose your input is a 300 by 300 color (RGB) image, and you are not using a convolutional network. If the first hidden layer has 100 neurons, each one fully connected to the input, how many parameters does this hidden layer have (including the bias parameters)?

27,000,100

9,000,100

27,000,001

9,000,001

$$(300 \times 300 \times 3 + 1) \times 100 = (270,000 + 1) \times 100 = 27,000,100$$

Suppose your input is a 256 by 256 grayscale image, and you use a convolutional layer with 128 filters that are each 3×3 . How many parameters does this hidden layer have (including the bias parameters)?

3584

1280

1152

75497600

$$(3 \times 3 \times 1 + 1) \times 128 = (9 + 1) \times 128 = 1280$$

You have an input volume that is $63 \times 63 \times 16$, and convolve it with 32 filters that are each 7×7 , using a stride of 2 and no padding. What is the output volume?

$29 \times 29 \times 32$

$16 \times 16 \times 32$

$29 \times 29 \times 16$

$16 \times 16 \times 16$

$$\text{Output size: } ((n - f + 2p) / s) + 1 = [(63 - 7 + 2(0)) / 2] + 1 = 29 \text{ \# Filters: 32}$$

You have an input volume that is 15×15×8, and pad it using “pad=2”. What is the dimension of the resulting volume (after padding)?

19×19×12 **19×19×8** 17×17×10 17×17×8

$$(n + 2p) \times (n + 2p) \times c = (15 + 2(2)) \times (15 + 2(2)) \times 8 = 19 \times 19 \times 8$$

You have an input volume that is 63×63×16, and convolve it with 32 filters that are each 7×7, and stride of 1. You want to use a “same” convolution. What is the padding?

1 2 7 **3**

$$p = (f - 1) / 2p = (7 - 1) / 2 = 3$$

You have an input volume that is 66×66×21, and apply max pooling with a stride of 3 and a filter size of 3. What is the output volume?

22 × 22 × 21

21 × 21 × 21

22 × 22 × 7

66 × 66 × 7

$$\text{Output size: } ((n - f) / s) + 1 = ((66 - 3) / 3) + 1 = 22 \text{ Filters: } 21$$

Which of the following are hyperparameters of the pooling layers? (Choose all that apply)

Average weights.

Filter size.

Number of filters.

Whether it is max or average.

In lecture we talked about “parameter sharing” as a benefit of using convolutional networks. Which of the following statements about parameter sharing in ConvNets are true? (Check all that apply)

It allows parameters learned for one task to be shared even for a different task (transfer learning).

It allows gradient descent to set many of the parameters to zero, thus making the connections sparse.

It allows a feature detector to be used in multiple locations throughout the whole input image/input volume.

It reduces the total number of parameters, thus reducing overfitting.

The sparsity of connections and weight sharing are mechanisms that allow us to use fewer parameters in a convolutional layer making it possible to train a network with smaller training sets. True/False?

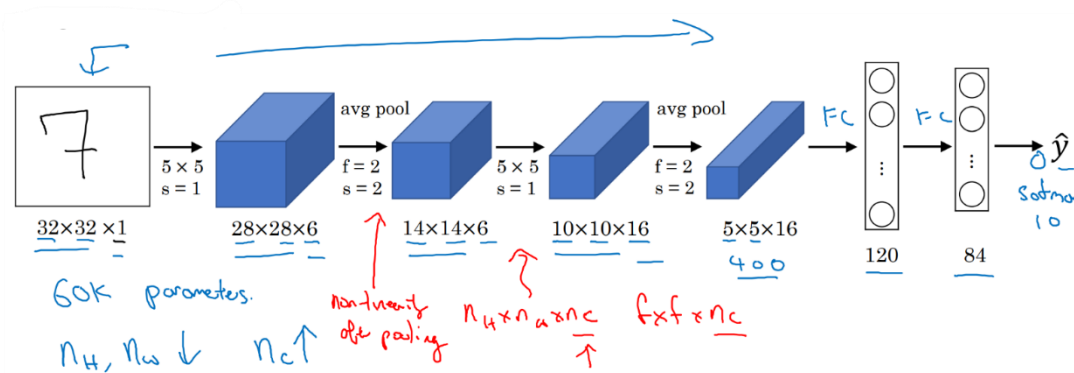
False

True

Deep Convolutional Models: Classic Networks

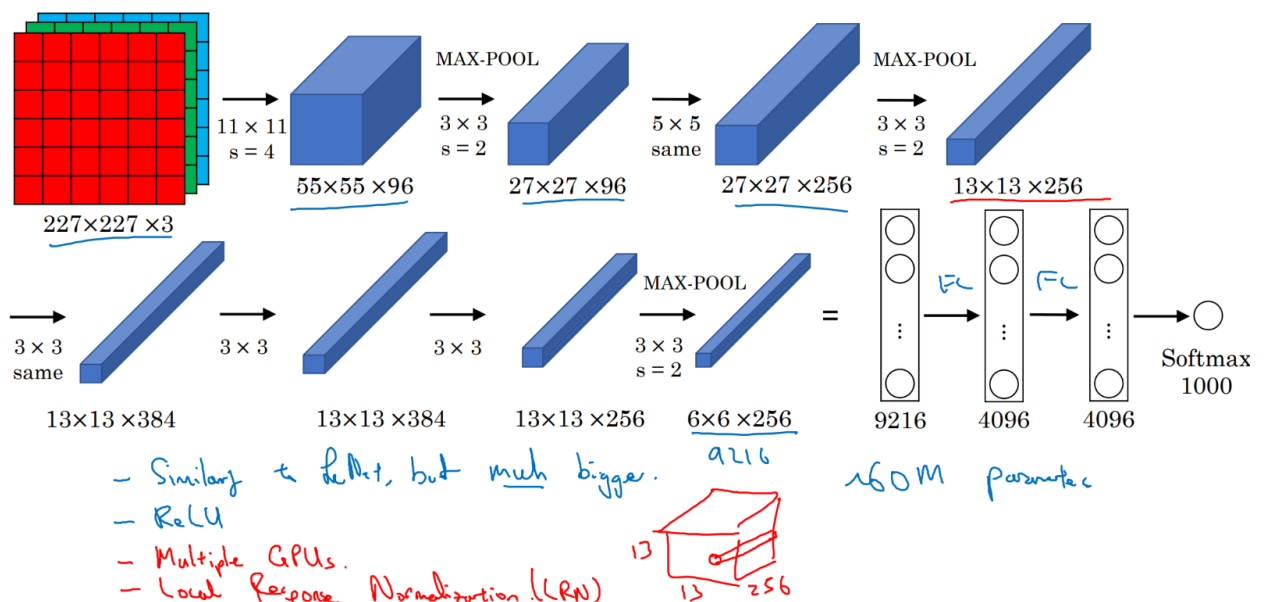
LeNet-5

- Recognizing hand-written digits from grayscale images
- Final softmax output (used an archaic equivalent at the time) for digit from 0-9
- Avg pooling no padding; 60,000 parameters; high and width decrease but number of channels increase deeper into NN; conv pool conv pool fc fc output
- Non-linearity after pooling layer; had complicated filter application that isn't necessary today



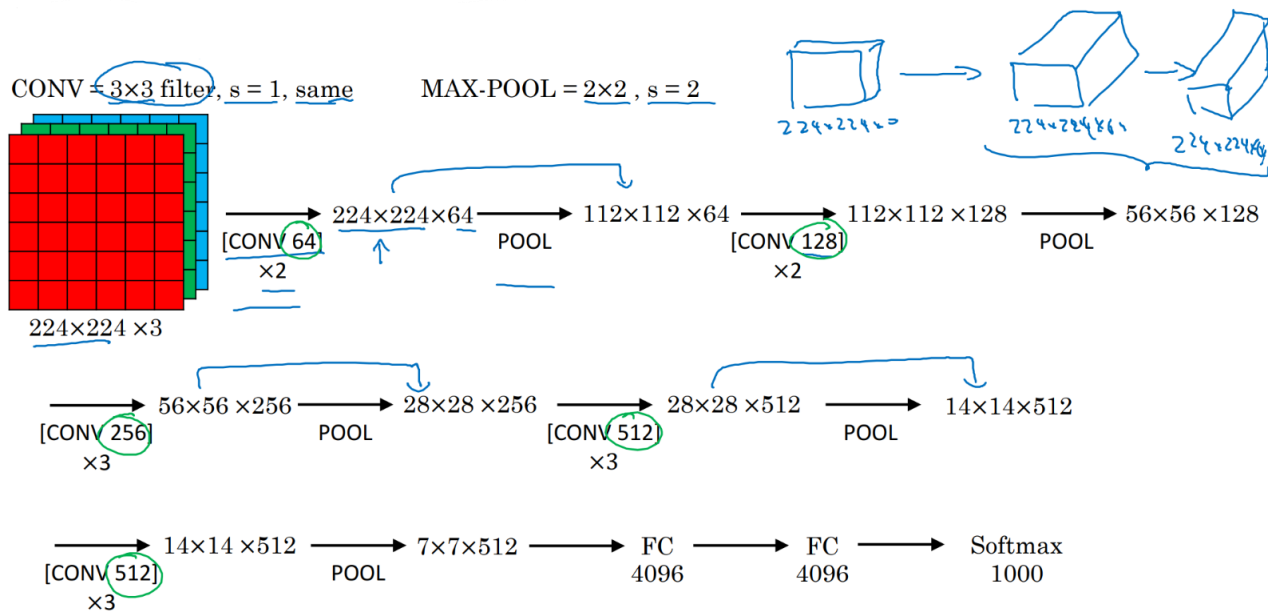
AlexNet

- Three channel images $\rightarrow$ 96 11×11 filters with stride 4
- Similar to LeNet but much bigger (60 mil parameters), used ReLU
- Multiple GPUs (slower at the time), archaic local response normalization layer (at one position of height and width, normalize numbers across all channels)



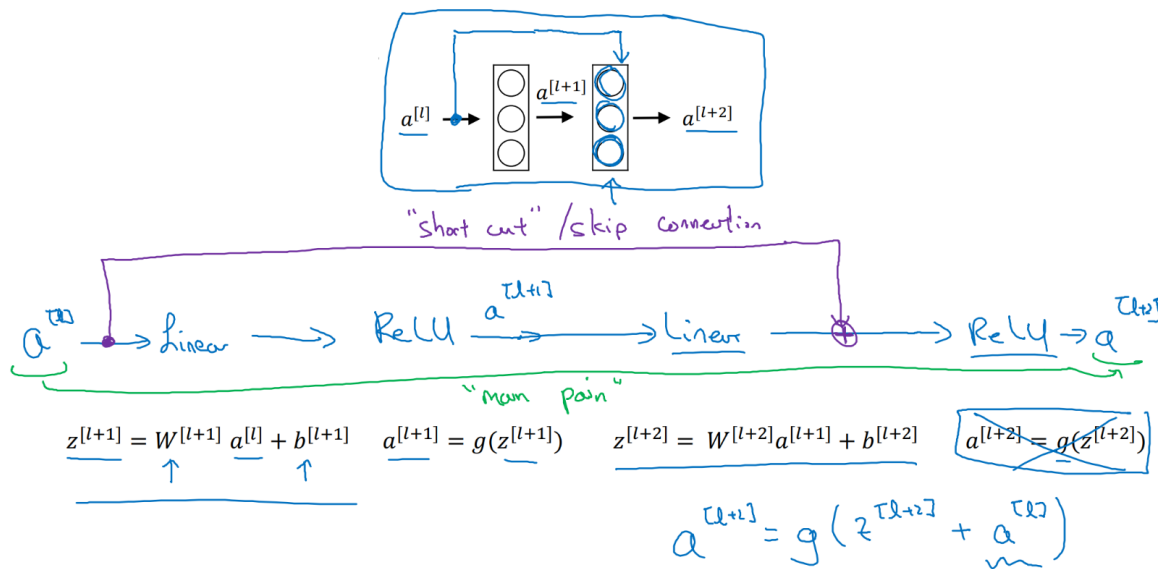
- **VGG (VGG-16)**

- Simpler architecture where you focus on having conv layers that are just 3x3 filters with stride 1 and same padding and 2x2 max pooling layers with stride 2
- 16 layers with weights; 138 mil parameters
- Height and width halves, number of filters doubles
- VGG-19 also exists

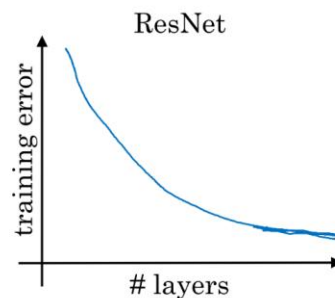
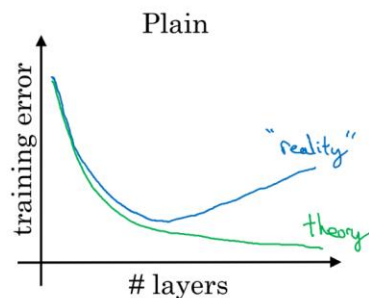
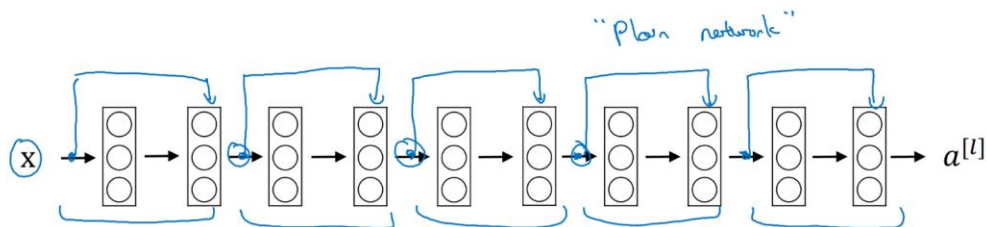


Residual Networks (ResNets)

- Very deep neural networks are difficult to train due to vanishing and exploding gradients
- **Skip connections** allow you to take activations from one layer and feed into another layer much deeper in the neural network
- **Residual block**
 - Main path: $a^{[l]}$ feeds into linear which feeds into ReLU, etc
 - Skip connection: Add $a^{[l]}$ before applying the next non-linearity (ReLU)
 - Using residual blocks allows you to train much deeper NNs

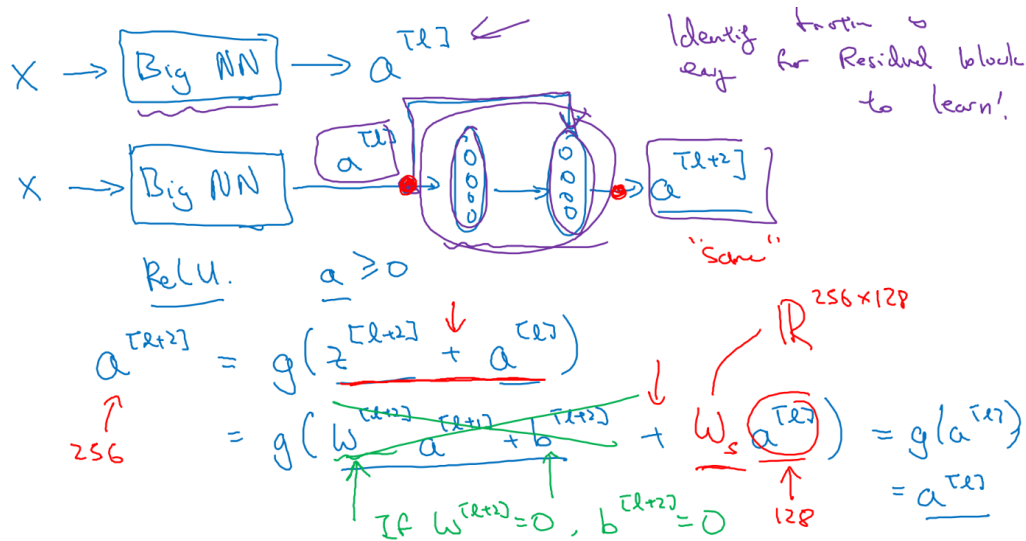


- Residual network
 - Stack several residual blocks together to form a deep network
 - Add skip connections to plain network

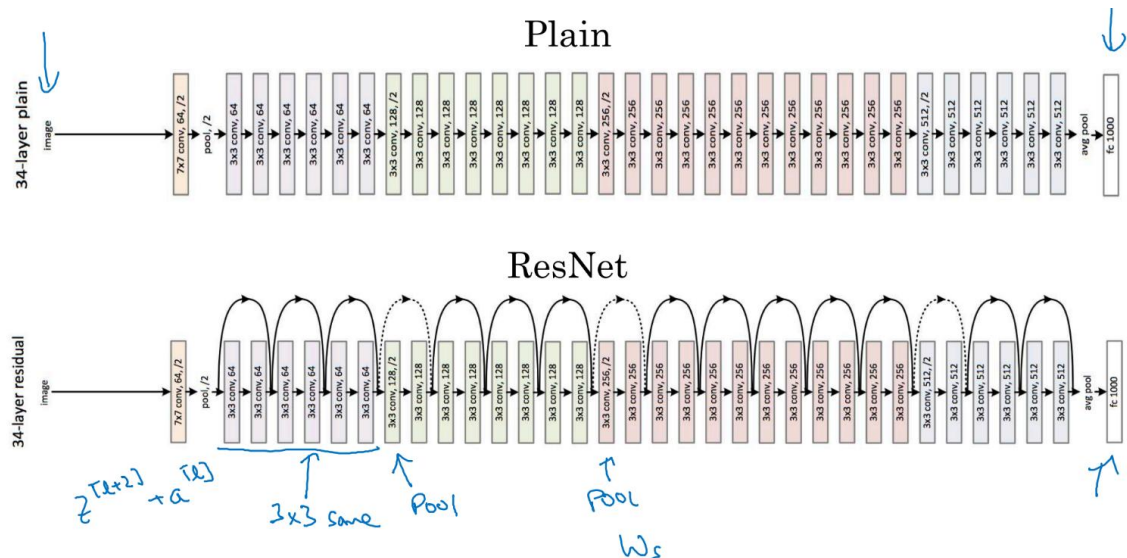


- Why ResNets work

- If using L2 regularization, that will tend to shrink the value of $W^{[l+2]}$. If = 0, you are left with only $g(a^{[l]}) = a^{[l]}$ (ReLU of non-negative value is the value)
- Identity function is easy for residual block to learn (hard for plain deep NNs)
- Adding two extra layers at middle/end of big NN doesn't hurt its performance. If they learn something useful, you can do even better than learning the identity fn
- Assumes $z^{[l+2]}$ and $a^{[l]}$ are the same dimensions $\rightarrow$ use "same" convolutions or you can add an extra matrix W_s to fix the dimension of $a^{[l]}$

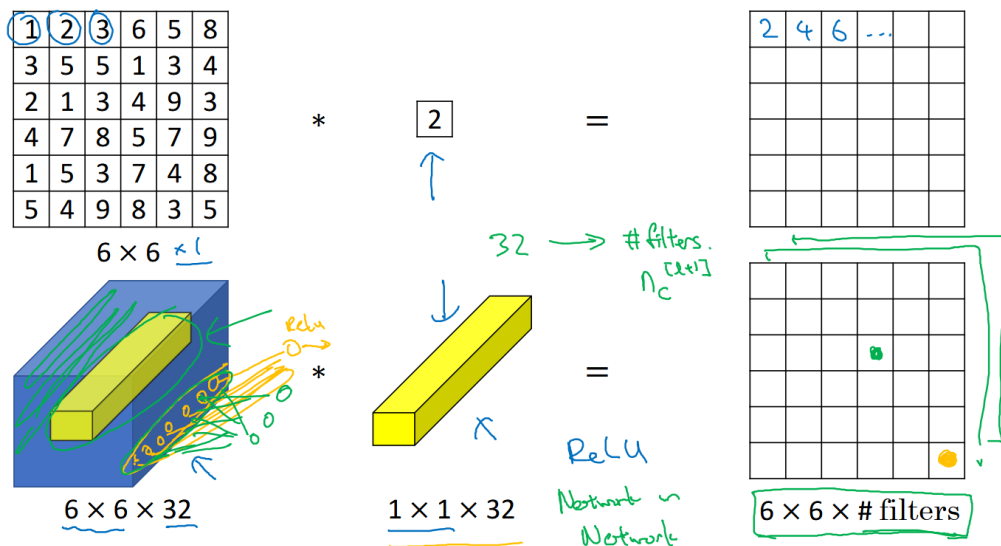


- 3x3 same convolutions preserve dimensions
- Conv conv conv pool ... FC $\rightarrow$ softmax

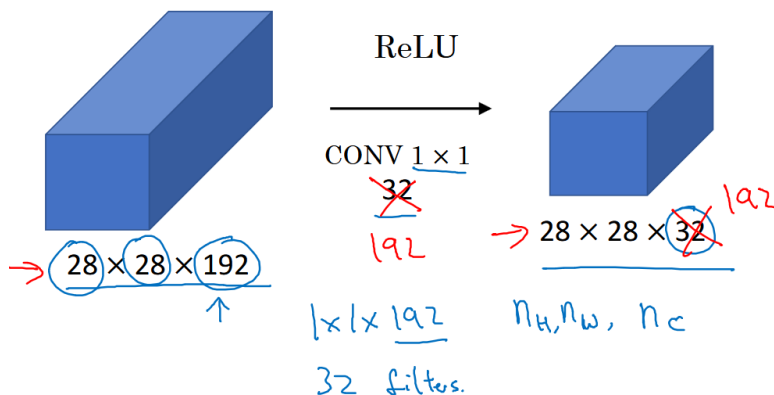


Network in Network and 1x1 Convolutions

- What does a 1×1 convolution do?
 - Looks at each of the $h \times w$ positions and take the element-wise product between the 32 numbers on the left and the 32 numbers in the filter, and then apply a ReLU
 - One slice through volume (one neuron that takes 32 #s as input, multiplies each of the 32 #s in one slice/in the same position by 32 weights, applying a ReLU non-linearity to it, and then outputting the corresponding result)
 - Multiple filters: multiple neurons taking the #s in one slice as inputs and building them up into an output
 - Fully connected neural network that applies to each of the 62 different positions and inputs 32 #s and outputs # filters $n_c^{[l+1]}$

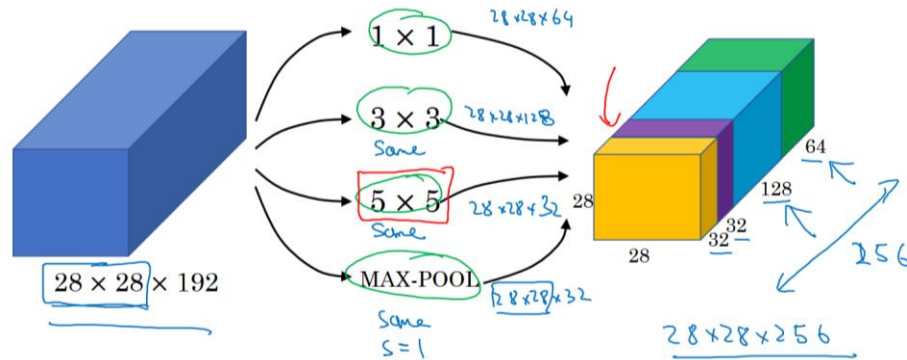


- Using 1×1 convolutions
 - Shrink # channels** by using 32 $1 \times 1 \times 192$ filters and output $28 \times 28 \times 32$
 - If not shrinking # channels, just applies non-linearity (complexity)

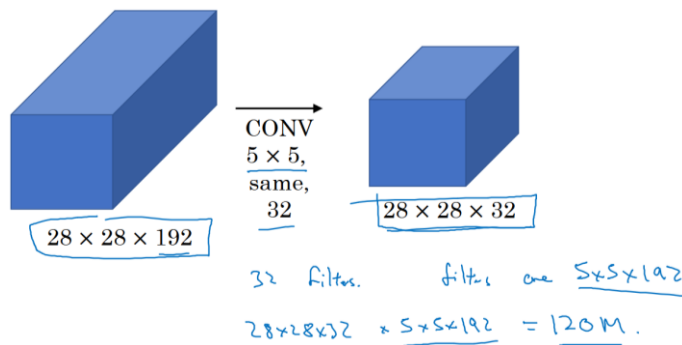


Inception Network

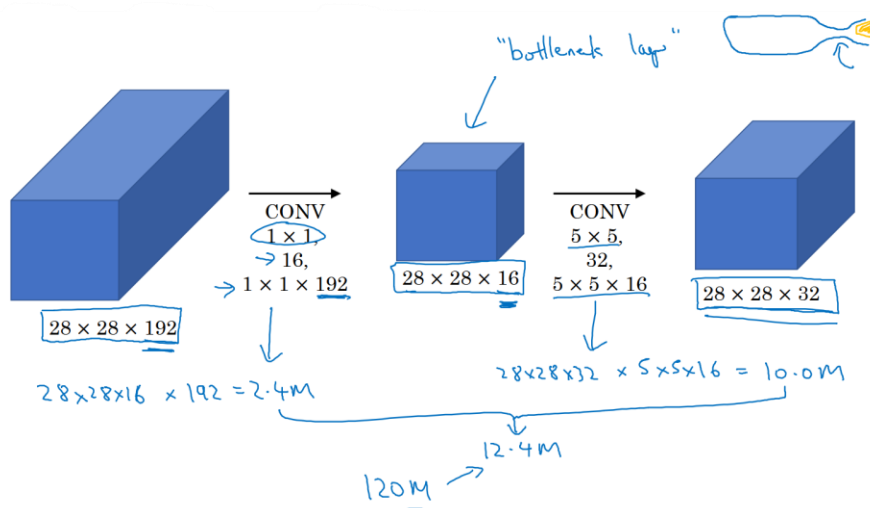
- Normally you have to pick what layer dimensions/type. Inception does them all and concatenates the results
- Let the network learn whatever parameters it wants to use (combinations of filter sizes)
- Pooling needs padding to concatenate with the others



- Computational cost

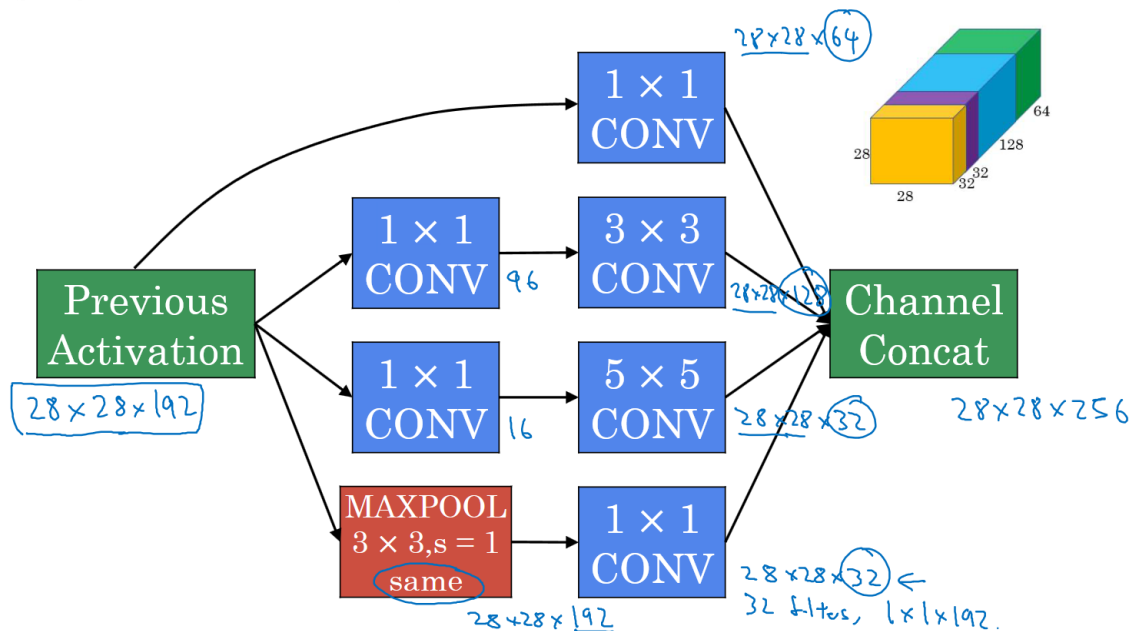


- Using 1×1 convolution
 - Much smaller intermediate volume (bottleneck layer), then apply bigger conv

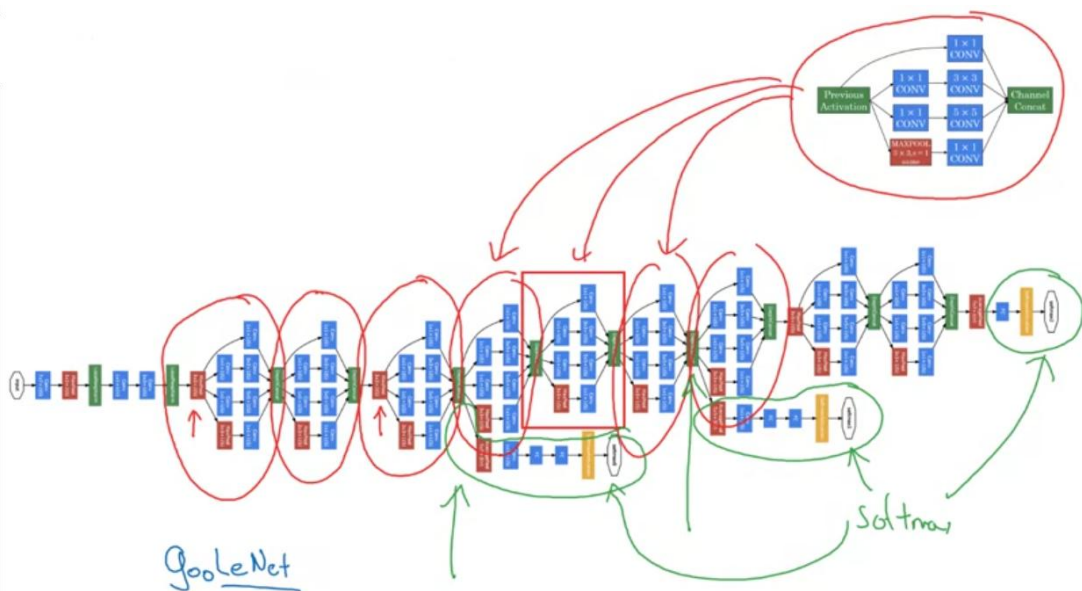


$1/10^{\text{th}}$ as many

- Inception module
 - Even with padding, pooling needs a 1×1 conv to shrink the # of channels



- Inception network
 - Puts inception modules together, many repeated blocks
 - Side branches use some hidden layer to try to make predictions. Helps ensure feature computed, even in hidden units in intermediate layers, aren't too bad for predicting the output. Has a regularizing effect on the inception network and helps prevent overfitting



MobileNet

- Low computational cost at deployment
- Useful for mobile and embedded vision applications
- Key idea: Normal vs. depthwise-separable convolutions
- Normal Convolution

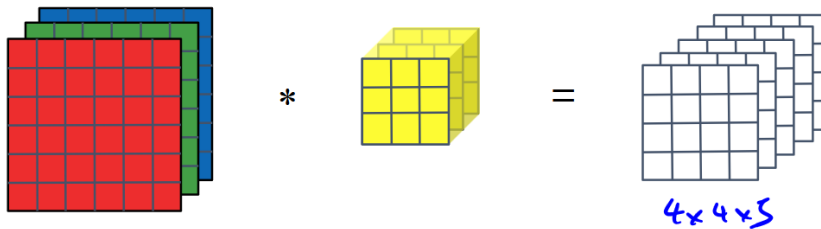
Computational cost = $\frac{\text{\#filter params}}{3 \times 3 \times 3} \times \frac{\text{\#filter positions}}{4 \times 4} \times \text{\# of filters}$

2160 = $3 \times 3 \times 3 \times 4 \times 4 \times 5$

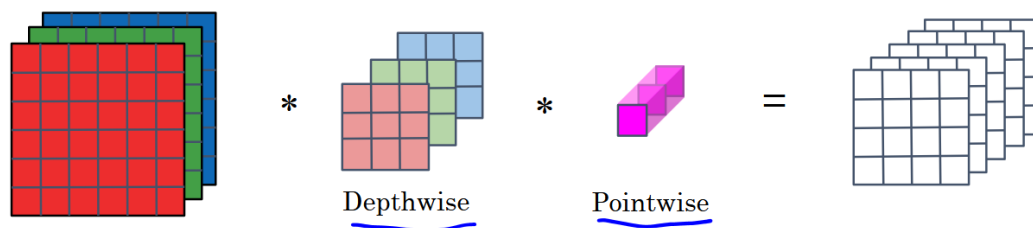
Depthwise Separable Convolution

- Two steps: depthwise convolution, then pointwise convolution

Normal Convolution

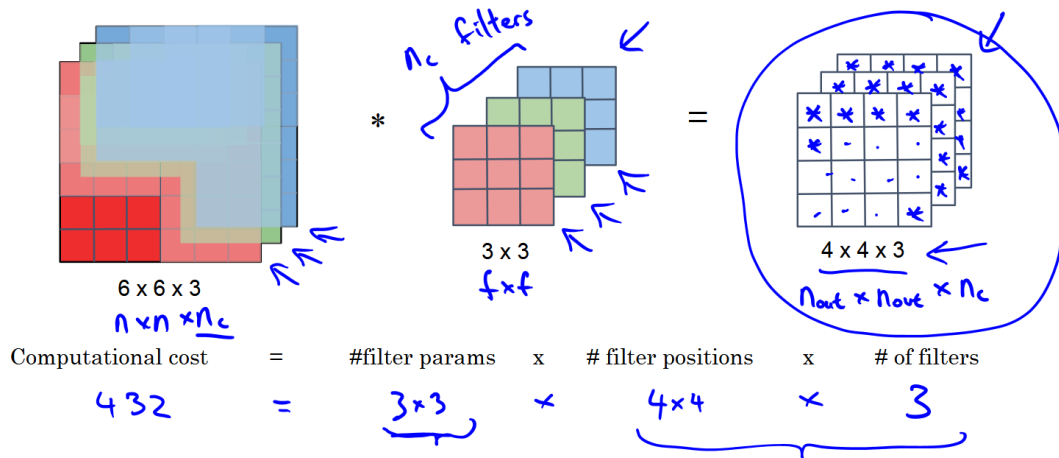


Depthwise Separable Convolution



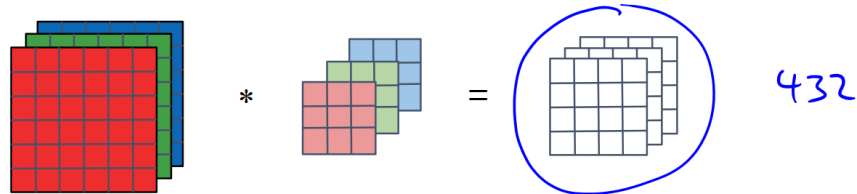
- **Step 1: Depthwise Convolution**

- Apply one of each of the filters to one of each of the input channels
- Red filter applies to 1st channel of input (9 multiplications), green to 2nd, etc

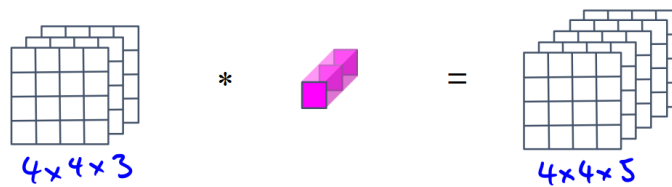


- Next, need to take this 4x4x3 intermediate value and carry out one more step to get the 4x4x5 output we want

Depthwise Convolution

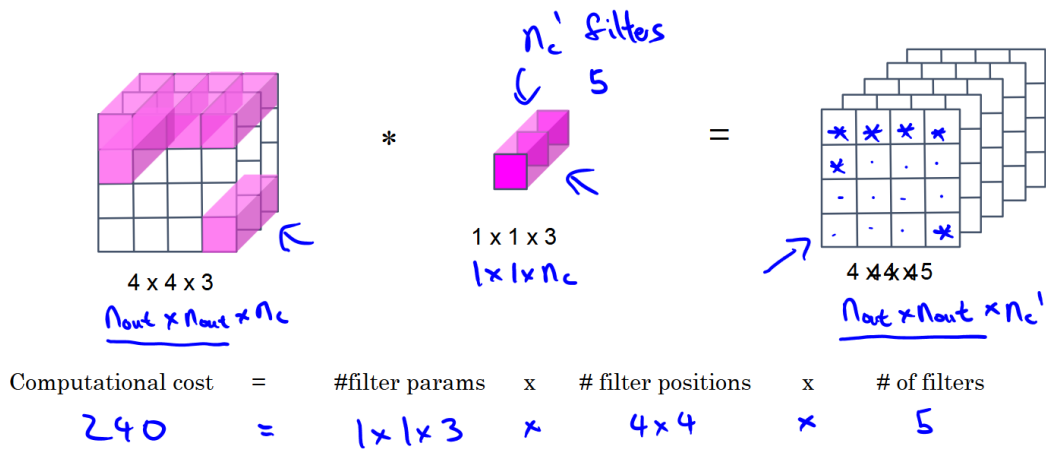


Pointwise Convolution



- Step 2: **Pointwise Convolution**

- Apply the $1 \times 1 \times 3$ filter to the same position in all channels of the input



- Cost Summary

- Depthwise separable was 31% as computationally expensive as normal conv
- Generally, DS is $1/n_c' + 1/f^2$ as computationally expensive as normal conv

Cost of normal convolution 2160

Cost of depthwise separable convolution

depthwise + pointwise
432 + 240 = 672

$\frac{672}{2160} = 0.31 \leftarrow$

$$= \frac{1}{n_c'} + \frac{1}{f^2}$$

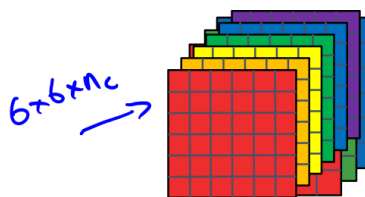
$$= \frac{1}{5} + \frac{1}{9}$$

$$= \frac{1}{512} + \frac{1}{3^2}$$

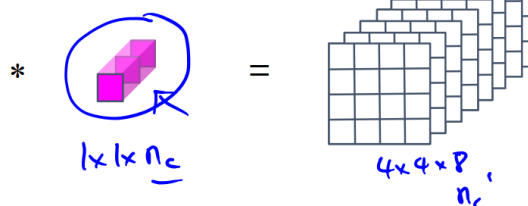
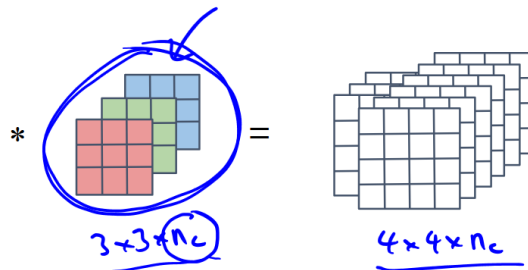
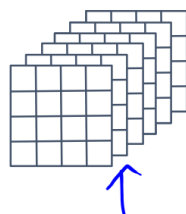
~ 10 times cheaper

- Depthwise separable convolution works for any number of input channels

Depthwise Convolution

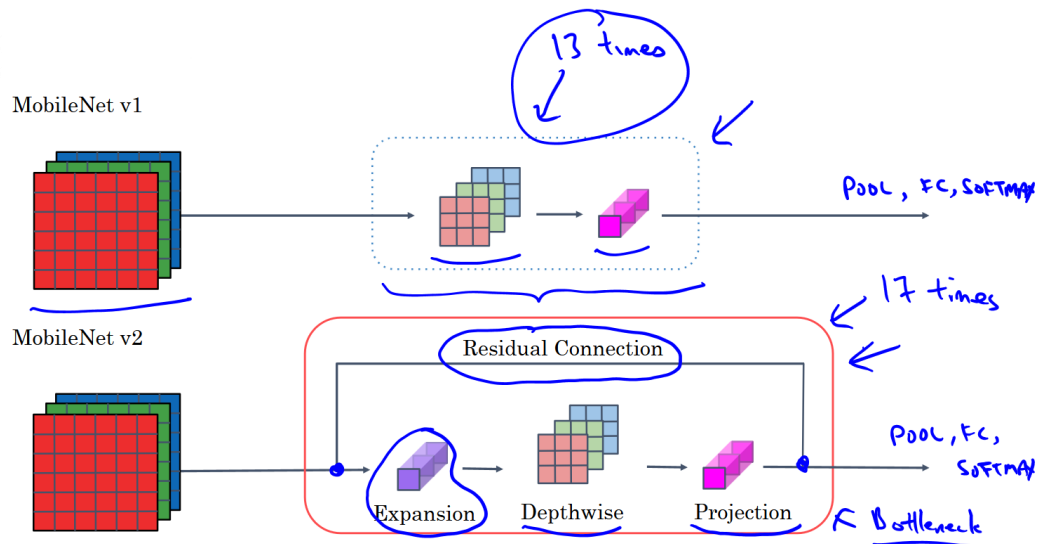


Pointwise Convolution

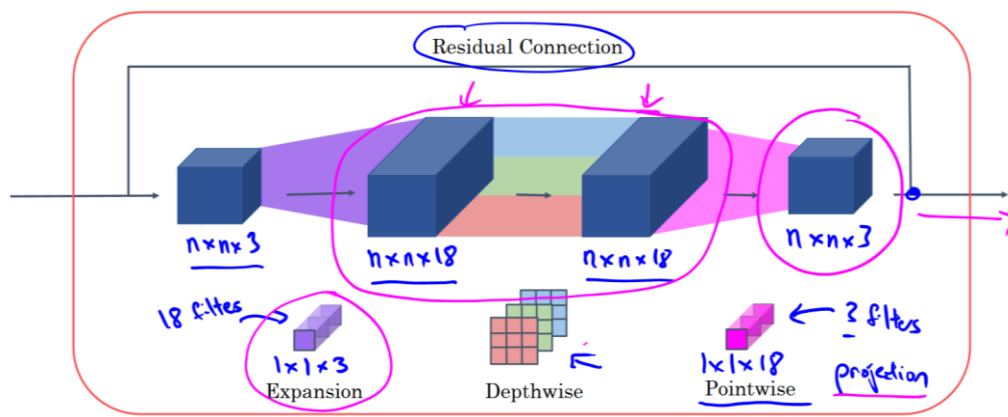


MobileNet Architecture

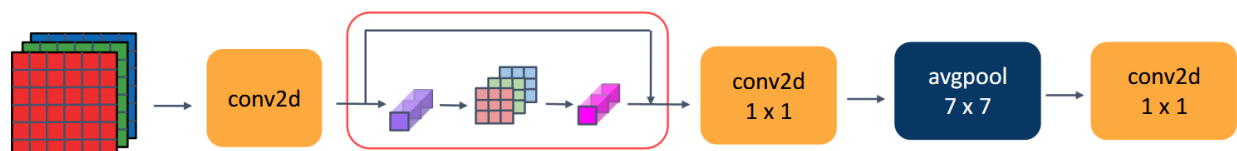
- MobileNet
 - Everywhere you would have used an expensive convolutional operation, instead use a less expensive depth wise separable convolution. Still performs well
 - V2 adds (a) residual/skip connections allowing for more efficient backprop and (b) expansion layer before DS



- MobileNet v2 bottleneck
 - Pass input directly to output and to expansion layer (commonly factor is 6)
 - Bottleneck block uses pointwise convolution (projection operation) to project back down to a smaller set of values so that amount of memory need to pass to the next block is reduced

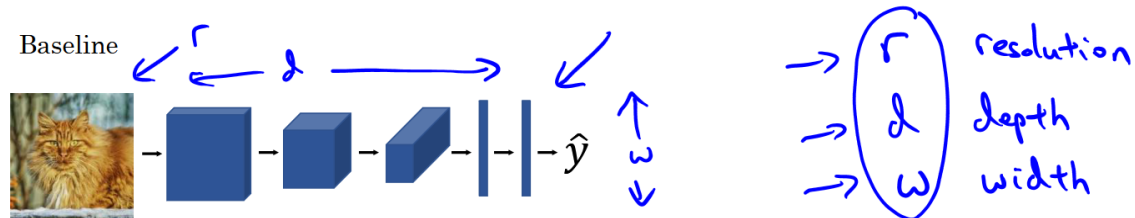


- MobileNet v2 Full Architecture



EfficientNet

- Tune MobileNet to a specific device/automatically scale NNs up or down
- Given a computational budget, what are good choices of resolution, depth, and width?
- **Compound scaling:** simultaneously scale up or down the resolution of the image and depth/width of the neural network

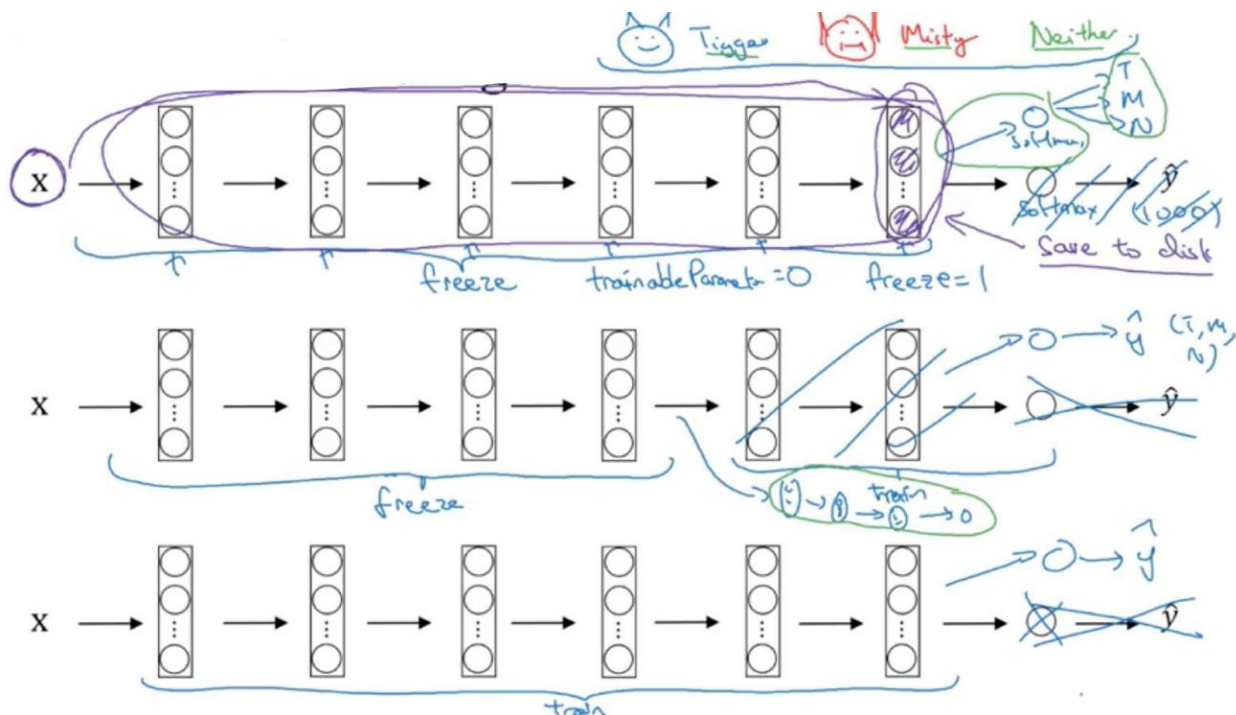


Using Open-Source Implementation

- Faster to use author's implementation and going from there than to reimplement from scratch
- Check and use GitHub. In command shell: `git clone [URL]`

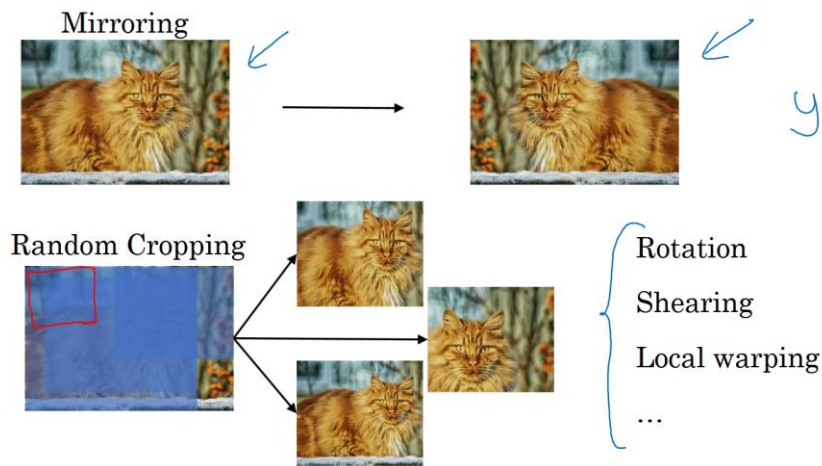
Transfer Learning

- No need to initialize from scratch; use online resources as pre-training to get going
- Download code/weights from a similar classifier. Replace softmax layer with your own
- Freeze parameters in downloaded network and just train the parameters associated with the softmax layer
- Other trick: there exists a function (fixed because you're not training it) that maps the input x to some set of activation in the last layer. Pre-compute that layer's activations and save them to disk. Training shallow softmax model from feature vector to make a prediction
- Larger training set: freeze fewer layers and train later layers or make new
- Huge training set: take open-source network/weights, use the whole thing just as initialization, and train the whole network

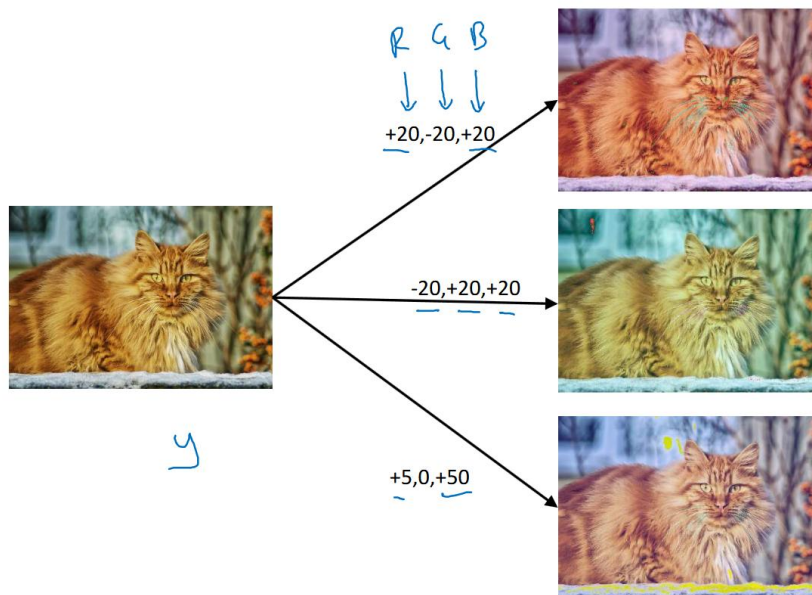


Data Augmentation

- More data to improve system performance (can't get enough data for computer vision)
- Common methods: mirroring, random cropping, rotation, shearing, local warping, etc.
- Note on random cropping: not truly random, some subsets are not useful. Just looking for reasonably large subsets of the actual image

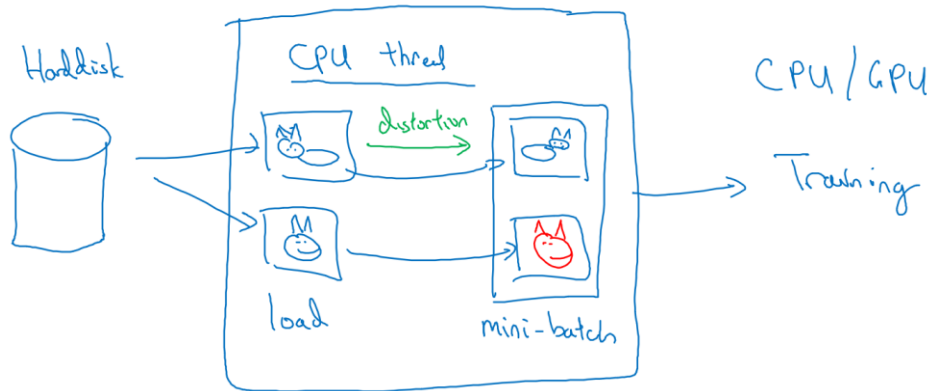


- Color shifting
 - Add distortions to R G and B channels
 - In practice you draw RGB from some probability distribution (could be smaller)
 - Makes algorithm more robust to changes in colors of images
 - Different way to sample RGB: PCA color augmentation



Advanced:
PCA
m2-class.org
[AlexNet paper
["PCA color augmentation."
R B G
~ ~ ~

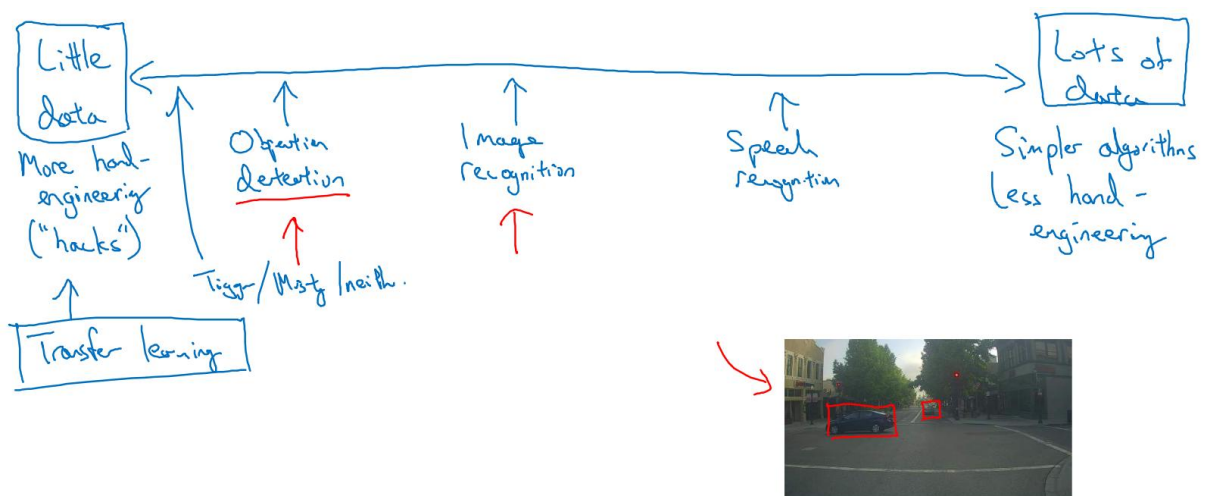
- Implementing distortions during training
 - CPU thread constantly loading images off hard disk
 - Use a CPU thread to implement distortions (form a mini-batch of data)
 - Data constantly pass to some other thread/process for implementing training
 - Separate threads for data and for training, can often run in parallel



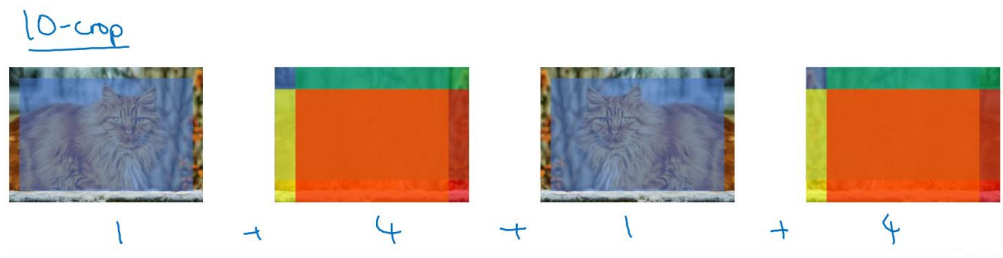
- Hyperparameters: how much color shifting, params for random cropping

The State of Computer Vision

- Two sources of knowledge:
 - Labeled data (x, y)
 - Lots of data → simpler algorithms and less hand-engineering
 - Hand engineering features/network architecture/other components
 - Little data → more hand-engineering, complex algorithms, transfer learning



- Tips for doing well on benchmarks/winning competitions
 - **Ensembling:** Train several networks (3-15) independently and average their outputs ($\hat{y}$). Requires lots of computer memory
 - **Multi-crop** at test time: run classifier on multiple versions of test images and average results. Not as much run memory, but slows run time
 - 10-crop method: crop center and 4 corners of image and its mirror



- Use open source code
 - Use architectures of networks published in the literature
 - Use open source implementations if possible
 - Use pretrained models and fine-tune on your dataset

When building a ConvNet, typically you start with some POOL layers followed by some CONV layers. True/False?

False True

Correct. It is typical for ConvNets to use a POOL layer after some Conv layers; sometimes even one POOL layer after each CONV layer; but is not common to start with POOL layers.

Which of the following do you typically see in a ConvNet? (Choose all that apply.)

FC layers in the last few layers

Multiple CONV layers followed by a POOL layer

Multiple POOL layers followed by a CONV layer

FC layers in the first few layers

LeNet - 5 made extensive use of padding to create valid convolutions, to avoid increasing the number of channels after every convolutional layer. True/False?

True **False**

The motivation of Residual Networks is that very deep networks are so good at fitting complex functions that when training them we almost always overfit the training data. True/False?

False True

Correct, very deep neural networks are hard to train and a deeper network does not always imply lower training error. Residual Networks allow us to train very deep neural networks.

The computation of a ResNet block is expressed in the equation:

$$a^{[l+2]} = g \left(\underbrace{W^{[l+2]}}_{\text{C}} g \left(\underbrace{W^{[l+1]} a^{[l]} + b^{[l+1]}}_{\text{A}} \right) + \underbrace{b^{[l+2]} + a^{[l]}}_{\text{B}} \right)$$

Which part corresponds to the skip connection?

The term in the red box, marked as C.

The equation of ResNet.

The term in the orange box, marked as B.

The term in the blue box, marked as A.

Which of the following equations captures the computations in a ResNet block?

$$a^{[l+2]} = g(W^{[l+2]} g(W^{[l+1]} a^{[l]} + b^{[l+1]}) + b^{[l+2]} + a^{[l]}) + a^{[l+1]}$$

$$a^{[l+2]} = g(W^{[l+2]} g(W^{[l+1]} a^{[l]} + b^{[l+1]}) + b^{[l+2]} + a^{[l]})$$

$$a^{[l+2]} = g(W^{[l+2]} g(W^{[l+1]} a^{[l]} + b^{[l+1]}) + b^{[l+2]}) + a^{[l]}$$

$$a^{[l+2]} = g(W^{[l+2]} g(W^{[l+1]} a^{[l]} + b^{[l+1]}) + b^{[l+2]})$$

Adding a ResNet block to the end of a network makes it deeper. Which of the following is true?

The performance of the networks is hurt since we make the network harder to train.

The number of parameters will decrease due to the shortcut connections.

It shifts the behavior of the network to be more like the identity function.

The performance of the networks doesn't get hurt since the ResNet block can easily approximate the identity function.

In a ResNet block the computations are given by $a^{[l+2]} = g(W^{[l+2]} a^{[l+1]} + b^{[l+2]} + a^{[l]})$ thus if $W^{[l+2]}$ and $b^{[l+2]}$ are zero then we get the identity function.

Which ones of the following statements on Residual Networks are true? (Choose all that apply.)

Using a skip-connection helps the gradient to backpropagate and thus helps you to train deeper networks

The skip-connection makes it easy for the network to learn an identity mapping between the input and the output within the ResNet block.

A ResNet with L layers would have on the order of L^2 skip connections in total.

The skip-connections compute a complex non-linear function of the input to pass to a deeper layer in the network.

Suppose you have an input volume of dimension $n_H \times n_W \times n_C$. Which of the following statements do you agree with? (Assume that the “1x1 convolutional layer” below always uses a stride of 1 and no padding.)

You can use a 2D pooling layer to reduce n_H , n_W , and n_C .

You can use a 1x1 convolutional layer to reduce n_C but not n_H and n_W .

You can use a 1x1 convolutional layer to reduce n_H , n_W , and n_C .

You can use a 2D pooling layer to reduce n_H , n_W , but not n_C .

For a volume of $125 \times 125 \times 64$ which of the following can be used to reduce this to a $125 \times 125 \times 32$ volume?

Use a POOL layer of size 2×2 with a stride of 2.

Use a POOL layer of size 2×2 but with a stride of 1.

Use a 1×1 convolutional layer with a stride of 2, and 32 filters.

Use a 1×1 convolutional layer with a stride of 1, and 32 filters.

Using 1×1 convolutions is a great way to reduce the depth dimension without affecting the other dimensions.

Which ones of the following statements on Inception Networks are true? (Choose all that apply.)

Inception networks incorporate a variety of network architectures (similar to dropout, which randomly chooses a network architecture on each step) and thus has a similar regularizing effect as dropout.

Making an inception network deeper (by stacking more inception blocks together) can improve performance, but can also lead to overfitting and increase in computational cost.

A single inception block allows the network to use a combination of 1x1, 3x3, 5x5 convolutions and pooling.

Inception blocks usually use 1x1 convolutions to reduce the input data volume's size before applying 3x3 and 5x5 convolutions.

Which of the following are true about bottleneck layers? (Choose all that apply)

By adding these layers we can reduce the computational cost in the inception modules.

Using the 1×1 convolutional layers we can reduce the depth of the volume and help reduce the computational cost of applying other convolutional layers with different filter sizes

Bottleneck layers help to compress the 1x1, 3x3, 5x5 convolutional layers in the inception network.

The bottleneck layer has a more powerful regularization effect than Dropout layers.

The use of bottlenecks doesn't seem to hurt the performance of the network.

significantly reduces the computational cost

Models trained for one computer vision task can't be used directly in another task. In most cases, we must change the softmax layer, or the last layers of the model and re-train for the new task. True/False?

False

True

When having a small training set to construct a classification model, which of the following is a strategy of transfer learning that you would use to build the model?

Use an open-source network trained in a larger dataset freezing the layers and re-train the softmax layer.

It is always better to train a network from a random initialization to prevent bias in our model.

Use an open-source network trained in a larger dataset, freeze the softmax layer, and re-train the rest of the layers.

Use an open-source network trained in a larger dataset. Use these weights as an initial point for the training of the whole network.

In Depthwise Separable Convolution you:

Perform two steps of convolution.

The final output is of the dimension $n_{out} \times n_{out} \times n_c$ (where n_c is the number of filters used in the pointwise convolution step).

For the “Depthwise” computations each filter convolves with only one corresponding color channel of the input image.

Perform one step of convolution.

The final output is of the dimension $n_{out} \times n_{out} \times n_c$ (where n_c is the number of color channels of the input image).

For the “Depthwise” computations each filter convolves with all of the color channels of the input image.

You convolve the input image with n_c number of $n_f \times n_f$ filters (n_c is the number of color channels of the input image).

You convolve the input image with a filter of $n_f \times n_f \times n_c$ where n_c acts as the depth of the filter (n_c is the number of color channels of the input image).

Which of the following are true about Depthwise-separable convolutions? (Choose all that apply)

The depthwise convolution convolves each channel in the input volume with a separate filter. Output of this kind of convolution is the same as the input

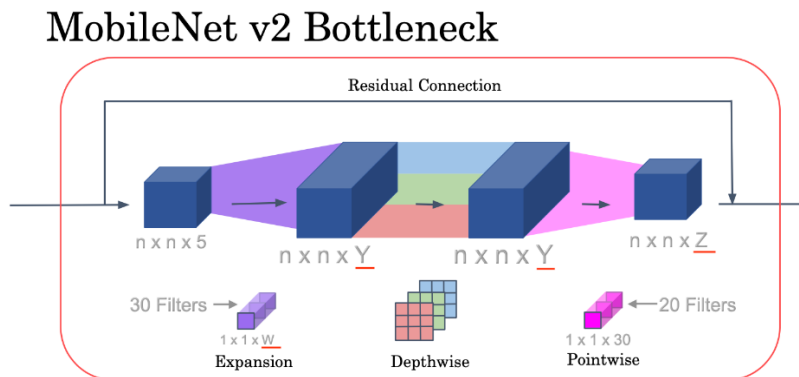
Depthwise-separable convolutions are composed of two different types of convolutions. Depthwise → Pointwise

The pointwise convolution convolves the output volume with 1×1 filters.

The number of filters for the output of the depthwise-separable convolution is determined by the number of 1×1 filters used.

The depthwise convolution convolves the input volume with 1×1 filters over the depth dimension.

Fill in the missing dimensions shown in the image below (marked W, Y, Z).



W = 5, Y = 20, Z = 5

W = 30, Y = 20, Z = 20

W = 30, Y = 30, Z = 5

W = 5, Y = 30, Z = 20

Suppose that in a MobileNet v2 Bottleneck block the input volume has shape $64 \times 64 \times 16$. If we use 32 filters for the expansion and 16 filters for the projection. What is the size of the input and output volume of the depthwise convolution, assuming a pad='same'?

$64 \times 64 \times 16, 64 \times 64 \times 32$

$32 \times 32 \times 32, 32 \times 32 \times 32$

$64 \times 64 \times 32, 64 \times 64 \times 32$

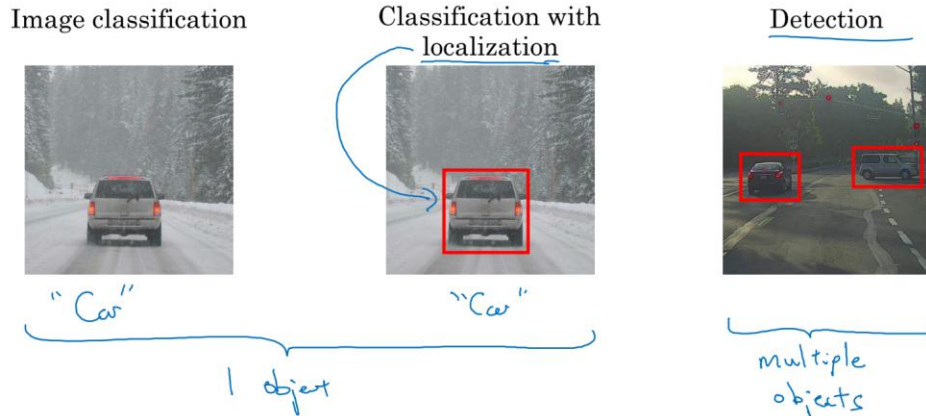
$64 \times 64 \times 32, 64 \times 64 \times 16$

$64 \times 64 \times 16 \rightarrow (1 \times 1 \text{ expansion, 32 filters}) \rightarrow 64 \times 64 \times 32 \rightarrow (\text{depthwise conv, same padding}) \rightarrow 64 \times 64 \times 32 \rightarrow (1 \times 1 \text{ projection, 16 filters}) \rightarrow 64 \times 64 \times 16$

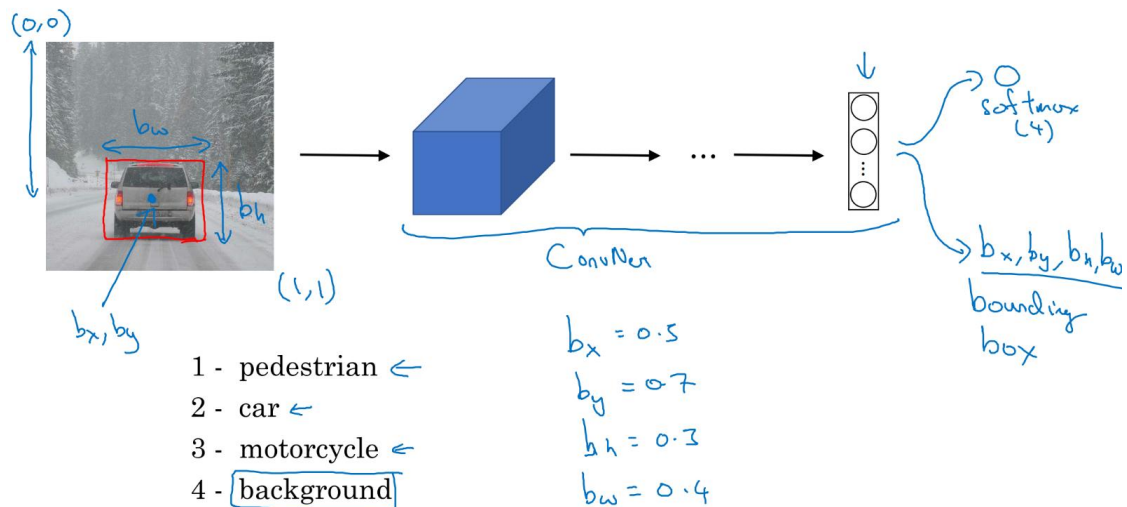
The size of the input and output volume of the depthwise convolution is determined by the number of filters in the expansion.

Object Localization

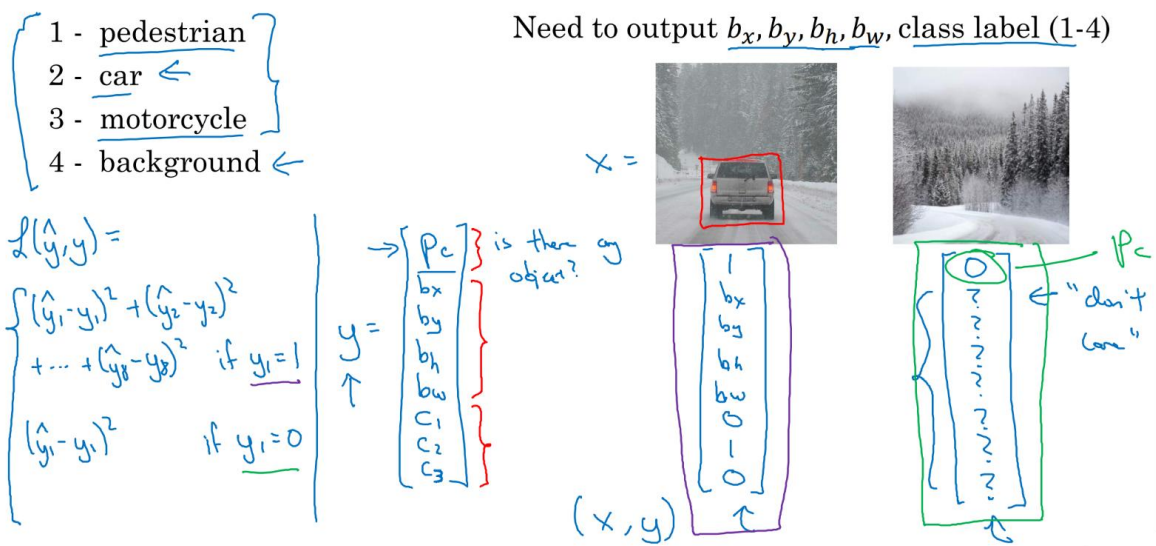
- **Localization:** putting a bounding box around the classified object
- **Detection:** detect and localize multiple objects in a picture



- Classification with localization
 - Softmax with 4 possible outputs (class of object)
 - Another output: 4 units of **bounding box** (b_x , b_y , b_H , b_W).
 - (b_x , b_y) is the mid-point, and b_H , b_W are the height and width of the bounding box

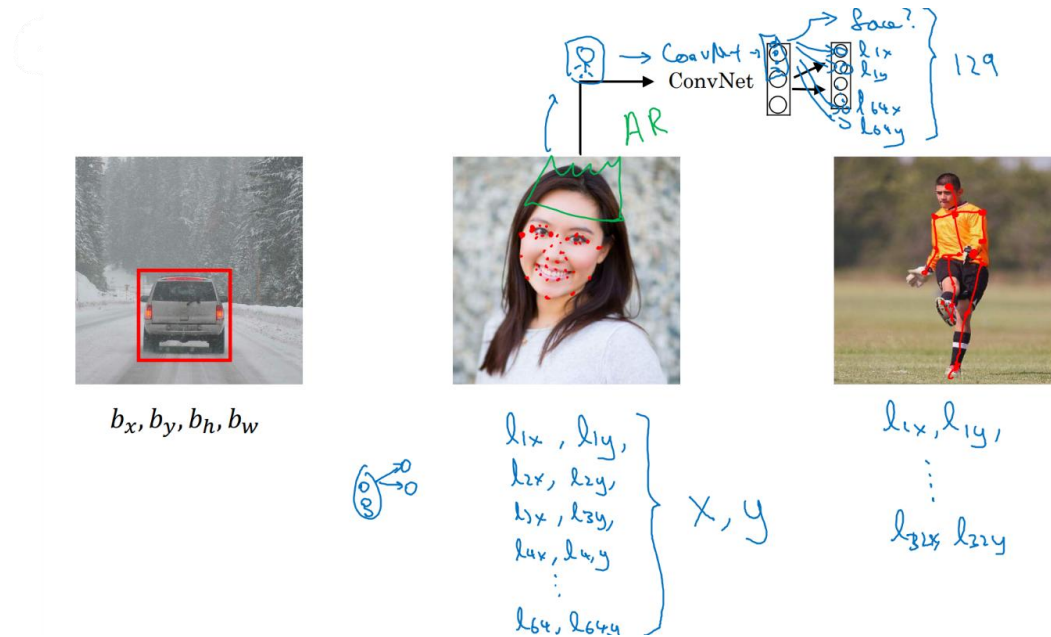


- Defining the target label y
 - Output y :
 - First component p_c : is there an object?
 - If there is an object, output the params of the bounding box b_x, b_y, b_h, b_w
 - If there is an object, output which class c_1, c_2, c_3 (one-hot encoded) (assuming at most one object here)
 - If no object, we don't care about the bounding box and class, so ? in y
 - Loss: $\sum (\hat{y}_i - y_i)^2$ (i = length of y) if $y_1 = 1$, or just $(\hat{y}_1 - y_1)^2$ if $y_1 = 0$
 - In practice, use log-like loss for c_1, c_2, c_3 to the softmax output, squared error for bounding box coordinates, and logistic regression loss for p_c



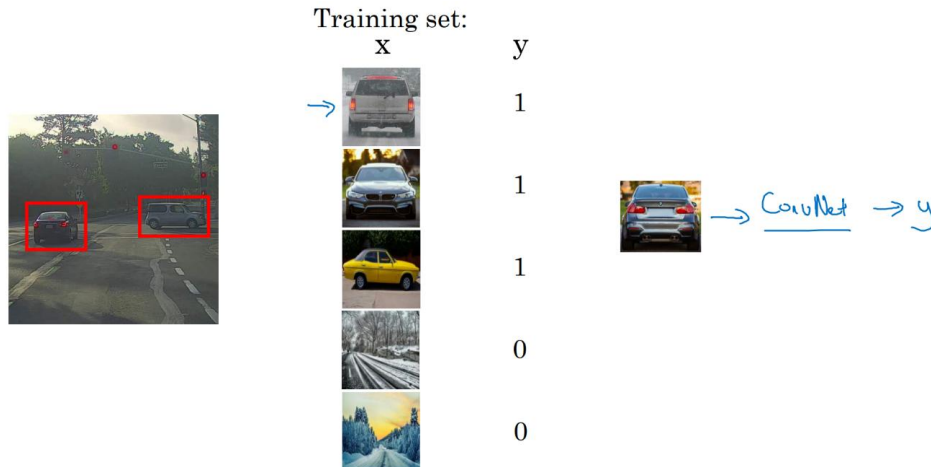
Landmark Detection

- In more general cases, you can have the NN just output x and y coordinates of important points in the image (**landmarks**)
- Landmark coordinates $\{l_{ix}, l_{iy}\} \rightarrow x, y$ = set of images and landmark annotations
- ConvNet outputting whether there is a face or not and coordinates of landmarks
- Landmark I needs to be consistent between images

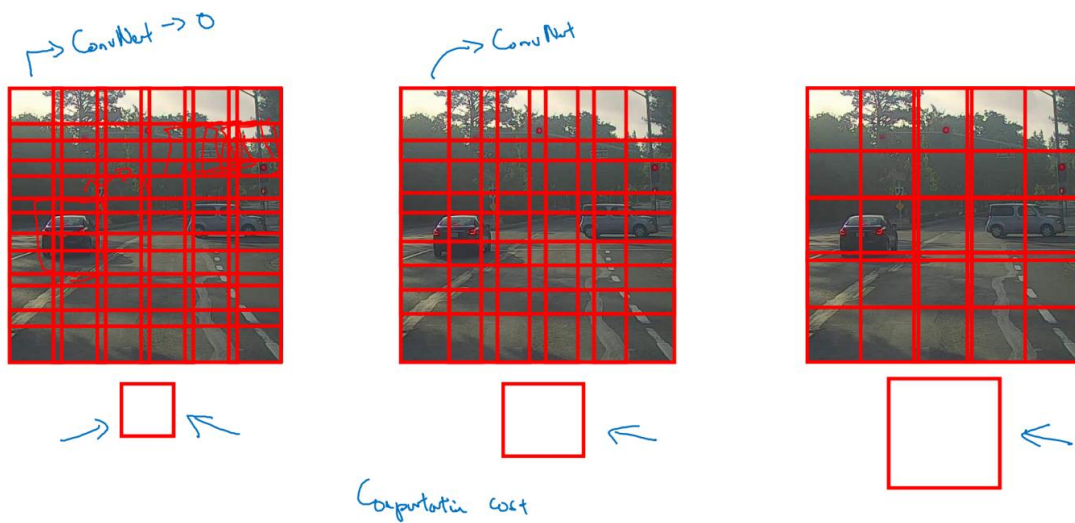


Object Detection

- Start by creating a labeled training set with closely cropped examples. Then create a ConvNet that take one of these images as input and outputs y (0 or 1, car or not)

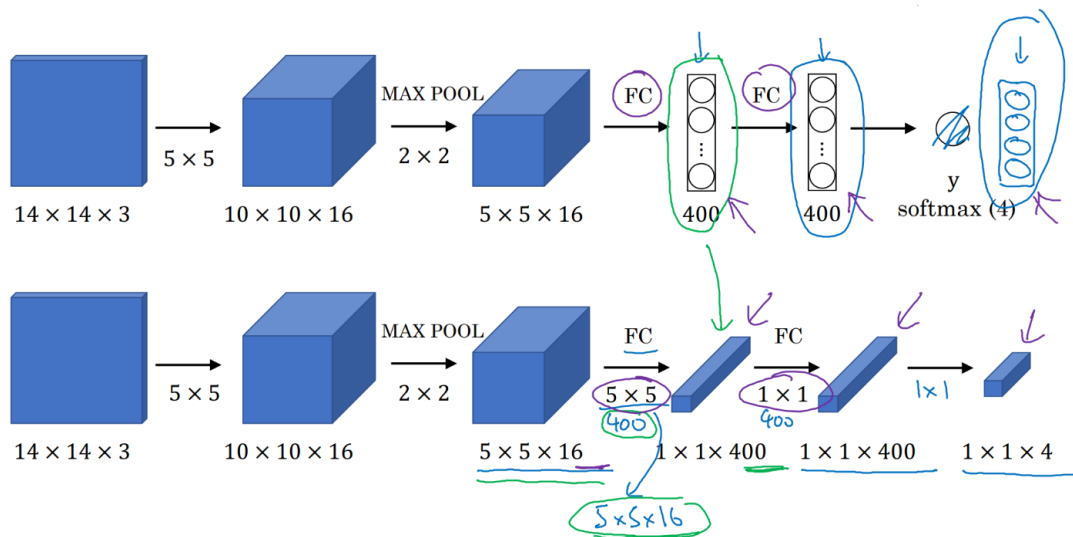


- Sliding windows detection algorithm**
 - Pick a certain window size. Input into ConvNet rectangular regions of that size (shifted across/down the whole image) and have it make predictions for each
 - If there is a car in the image, there will be a window where ConvNet will detect it
 - Disadvantage: computational cost of so many images
 - Stride will reduce # windows, but coarser granularity may hurt performance
 - Infeasibly slow or unable to localize accurately
 - Sliding window can be implemented convolutionally much more efficiently

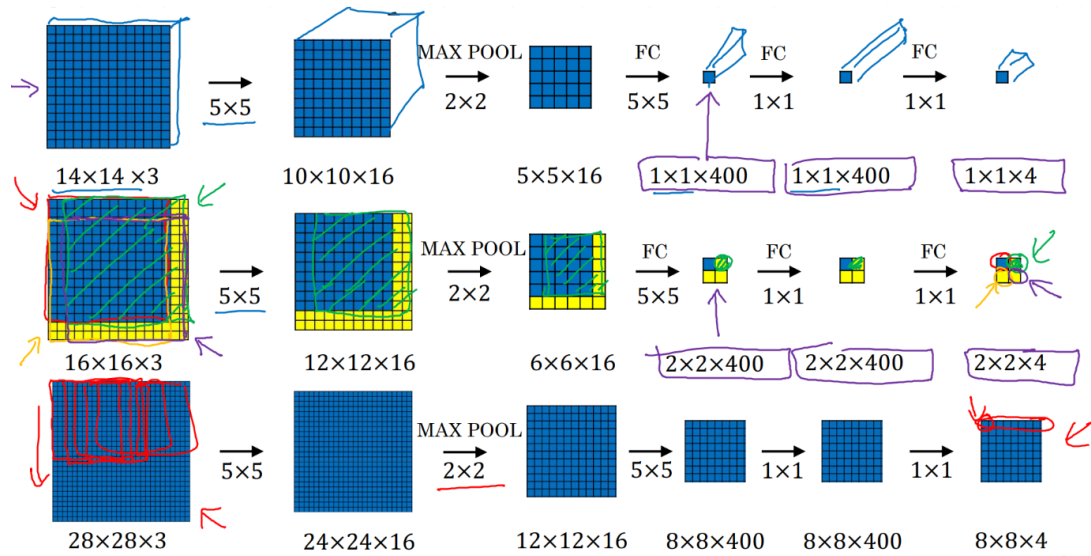


Convolutional Implementation of Sliding Windows

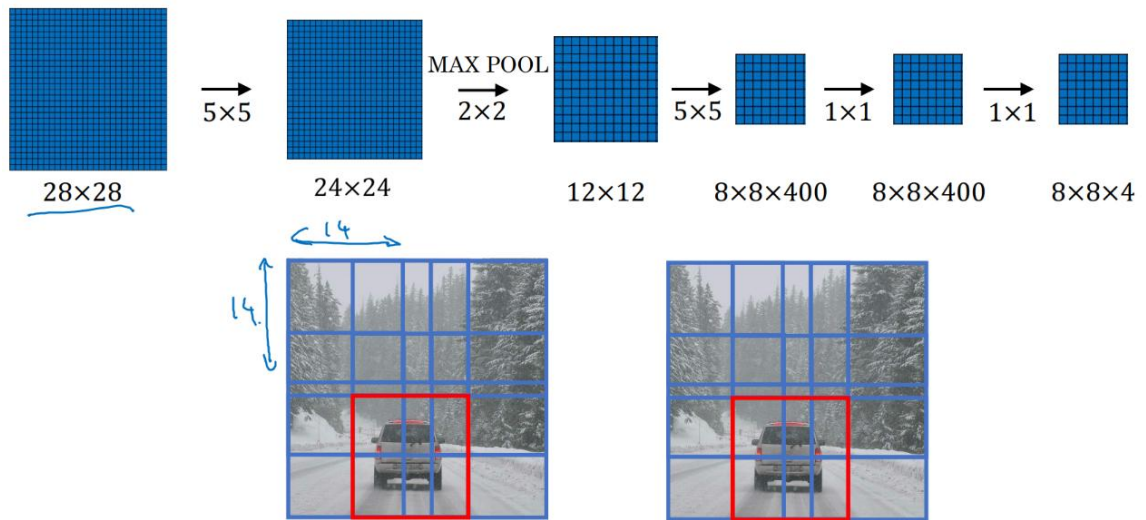
- Turning FC layer into convolutional layers
 - Softmax outputs probability of each class here
 - Convolve with 400 $5 \times 5 \times 16$ filters $\rightarrow 1 \times 1 \times 400$. Mathematically the same as a FC layer: each of the 400 nodes has a $5 \times 5 \times 16$ filter so is some arbitrary linear function of the $5 \times 5 \times 16$ activations from the previous layer
 - Then convolve with a 1×1 , which gives the same $1 \times 1 \times 400$ dimensions
 - One last 1×1 filter and softmax activation down to $1 \times 1 \times 4$



- Convolution implementation of sliding windows
 - If ConvNet inputs $14 \times 14 \times 3$ images but test set image is $16 \times 16 \times 3$ (yellow stripe)
 - Slide around by 2 to get all regions (top-left, top-right, bottom-left, bottom-right)
 - A lot of the computation done by the four ConvNets is highly duplicative
 - Conv sliding windows combines the forward passes of the ConvNet to share computation. Now you get a $2 \times 2 \times 400$ intermediate volume (instead of $1 \times 1 \times 400$) and a final output that is $2 \times 2 \times 4$ (instead of $1 \times 1 \times 4$). Each position of the 2×2 gives the result for the corresponding cube of image (top-left, top-right, bottom-left, bottom-right)



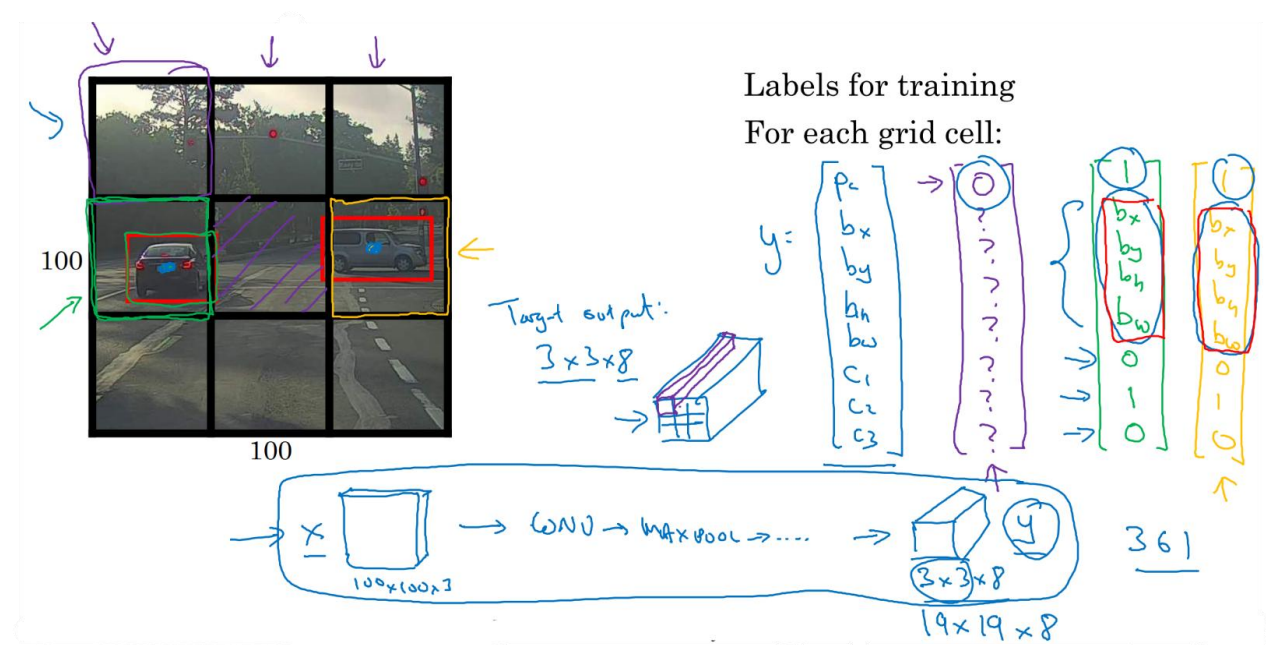
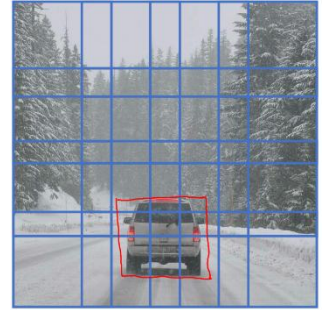
- Instead of sequentially, implement entire image and convolutionally make all predictions at the same time by one forward pass through the ConvNet



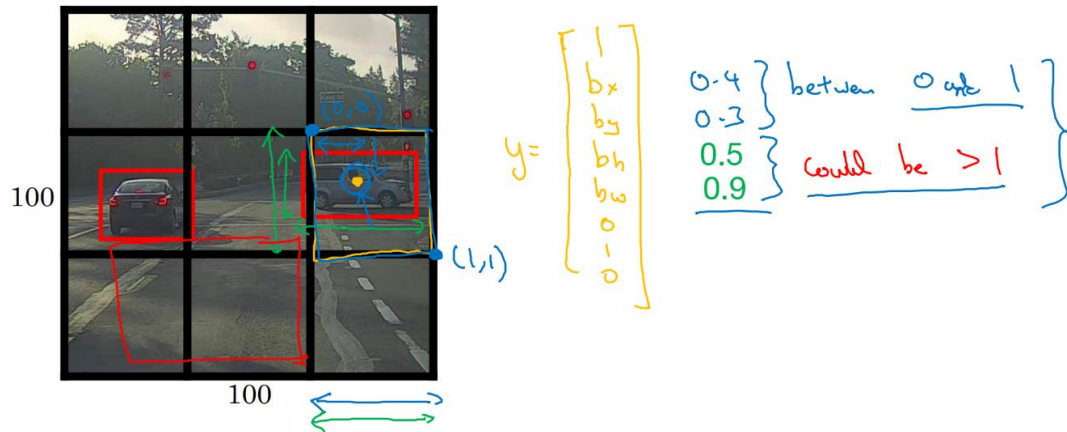
- Weakness: position of bounding boxes will not be too accurate

Bounding Box Predictions

- Output accurate bounding boxes
 - Perfect bounding box may not even be a perfect square
- **YOLO (You Only Look Once) algorithm**
 - Place a grid on image (usually finer, like 19x19) and apply image classification and localization algorithm to each cell of the grid. For each grid cell, specify a label $y = [p_c, \text{bounding params, classes}]^T$
 - Assigns each object to the grid cell containing the midpoint (doesn't count some sticking into the next cell)
 - Final output volume is $3 \times 3 \times 8$ (8D y vector)
 - Train network on x $100 \times 100 \times 3$. Choose conv and maxpool layers such that it eventually maps to a $3 \times 3 \times 8$ volume
 - Works okay as long as you don't have more than one object per cell (finer grid reduces chance of multiple objects per cell)
 - NN outputs bounding boxes of any aspect ratio with much more precise coordinates that aren't just dictated by the stride size of sliding windows classifier. Run the algorithm all at once, very efficient

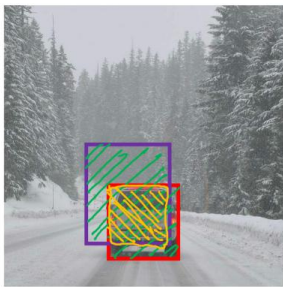


- Specify the bounding boxes
 - $0 < b_x, b_y < 1$ (orange dot is within bounds of grid it's assigned to)
 - b_H, b_W fraction of grid cell, can be > 1 (if bounding box spills into another cell)



Intersection Over Union (IOU)

- Used for evaluating and adding another component to an object detection algorithm
- Evaluating object localization
 - Ratio of areas (intersection / union)
 - Perfect = 1, 0.5 is most common, can be more stringent
 - Generally, IoU is a measure of the overlap between two bounding boxes



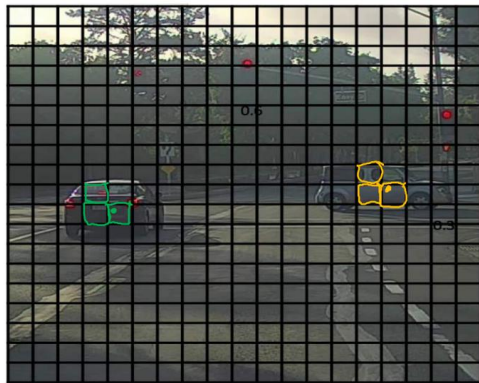
Intersection over Union

$$= \frac{\text{size of } \text{yellow box}}{\text{size of } \text{green box}}$$

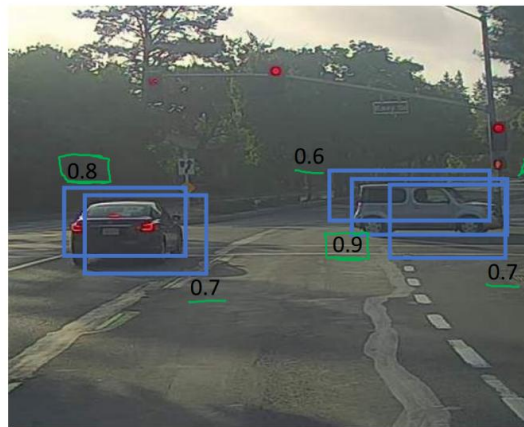
"Correct" if $\text{IoU} \geq 0.5$ ←
0.6 ←

Non-Max Suppression

- Non-max suppression ensures the algorithm only detects a given object once
- Example
 - Only one grid cell has the midpoint here, but in practice you're running an object classification and localization algorithm for every split cell. Several cells might think they have the center of the car in it. Non-max suppression cleans this.
 - Takes largest p_c , suppresses any of the rest with high overlap (IoU). Find the next highest p_c (likely a different object) and repeat. These are the final predictions



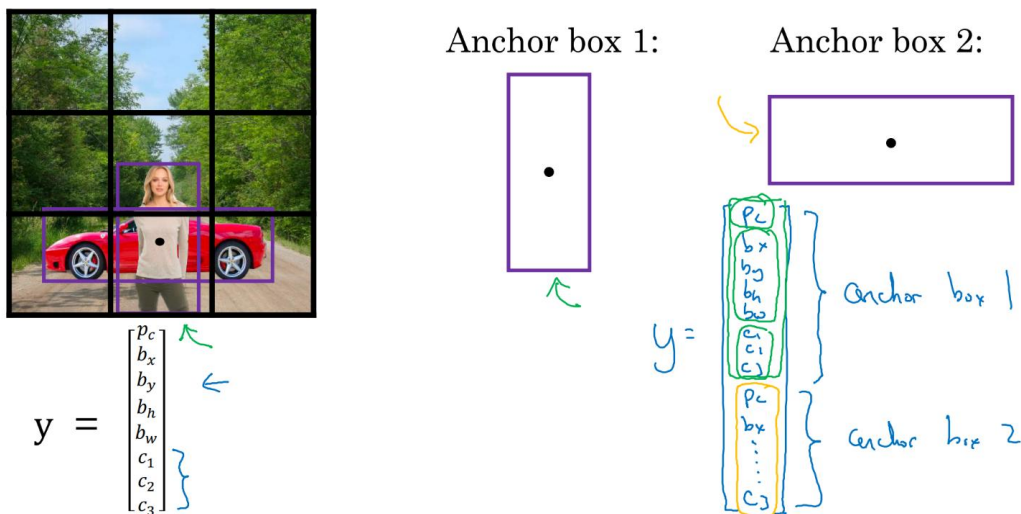
19x19



- Each (cell) output prediction is $[p_c \ b_x \ b_y \ b_H \ b_W]^T$
 - looking at one object class here. If you have more classes, independently carry out non-max suppression c times, one on each of the output classes
- Discard all boxes with $p_c \leq 0.6$.
- While there are any remaining boxes:
 - Pick the box with the largest $p_c \rightarrow$ output that as a prediction
 - Discard any remaining box with $\text{IoU} \geq 0.5$ with the box output in the previous step

Anchor Boxes

- What is a grid cell wants to detect multiple objects?
- Overlapping objects
 - Midpoint of pedestrian and car are in the same place
 - y vector can't output two detections, would have to pick one
 - Predefine different shapes (anchor boxes) and associate predictions with each
 - Pedestrian shape is similar to anchor box 1, car is similar to anchor box 2
 -



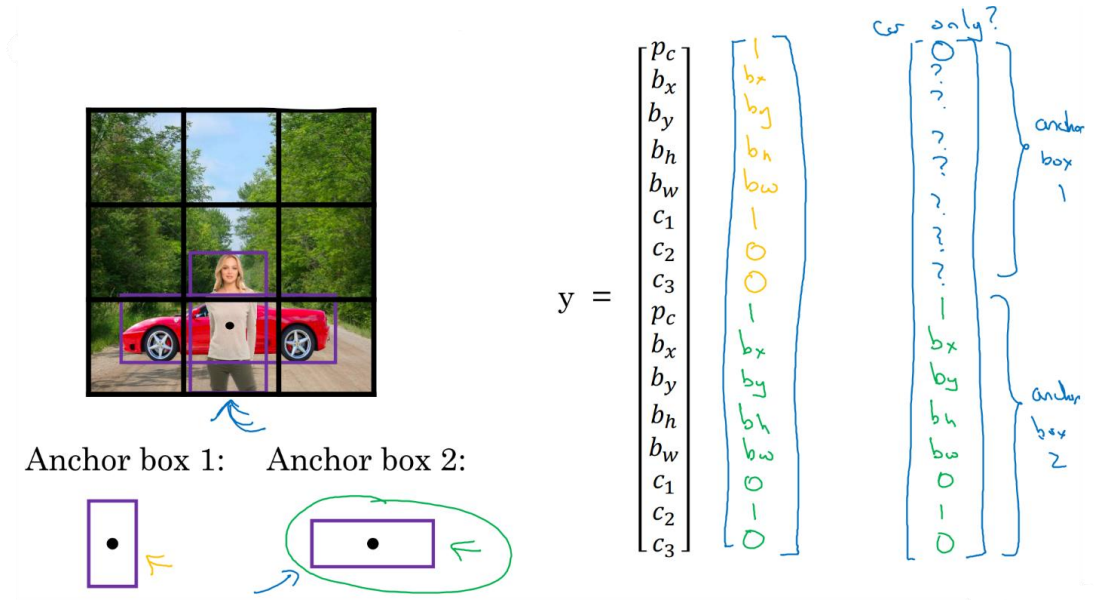
- Previously: Each object in training image is assigned to grid cell that contains that object's midpoint (output y: 3x3x8)
- With two anchor boxes: Each object in training image is assigned to grid cell that contains object's midpoint and anchor box for the grid cell with highest IoU
 - See which of the anchor boxes has a higher IoU with the ground truth bounding box. Whichever it is, that inject then gets assigned (grid cell, anchor box) pair

Output y:

$$3 \times 3 \times \underline{16}$$

$$3 \times 3 \times \underline{2 \times 8}$$

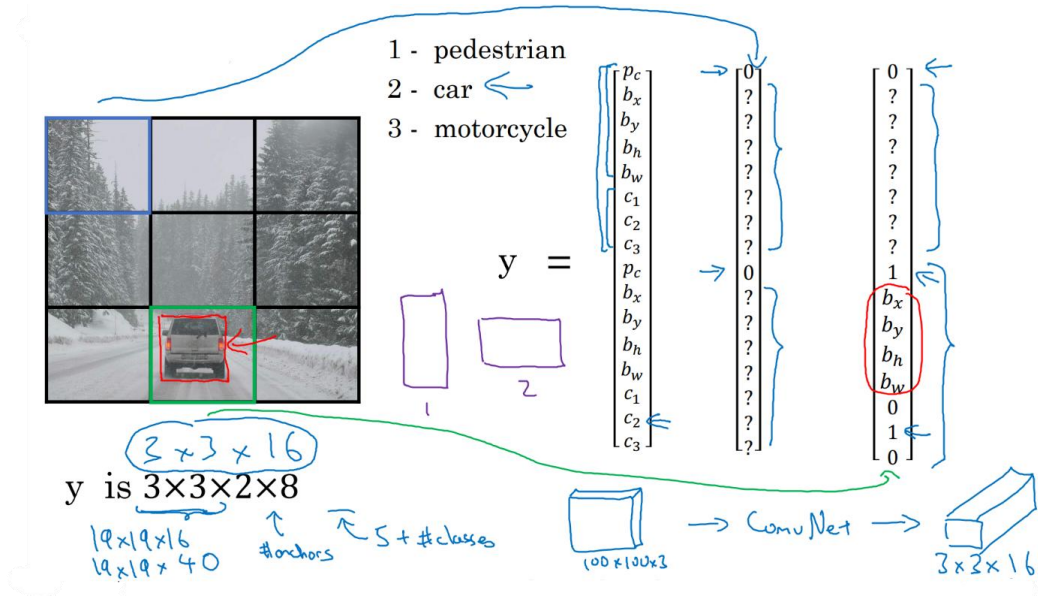
- Example
 - Pedestrian resembles anchor box 1 → assigned to top half of y vector, $c_1=0$
 - Car resembles anchor box 2 → assigned to bottom half of y vector, $c_2=1$
 - Car only: first anchor box is no object [0 ? ? ?]



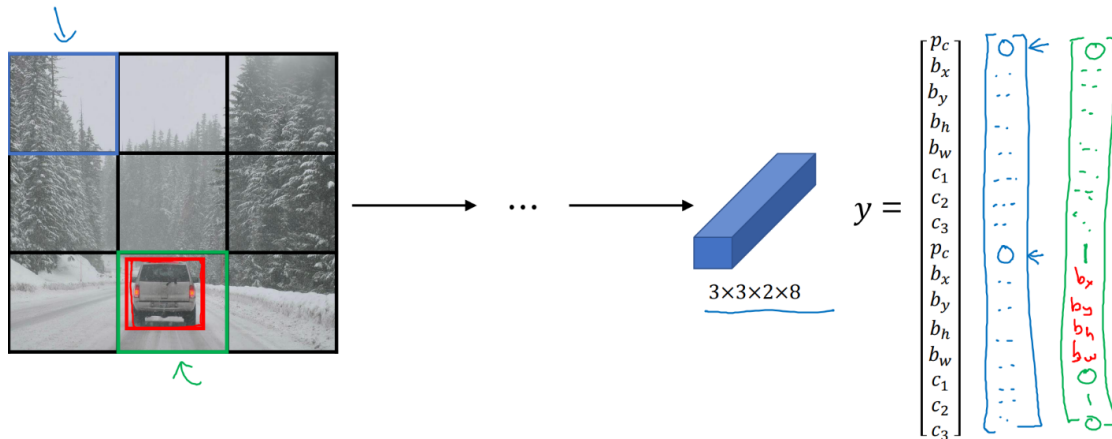
- Can't handle two anchor boxes but three objects in the same cell. Also can't handle two objects in the same cell with the same anchor box shape. You would have to implement a default tiebreaker
- K-means algorithm to group the types of objects shapes and use that to select a set of anchor boxes that is most stereotypically representative of the objects you're trying to detect

Putting It Together: YOLO Algorithm

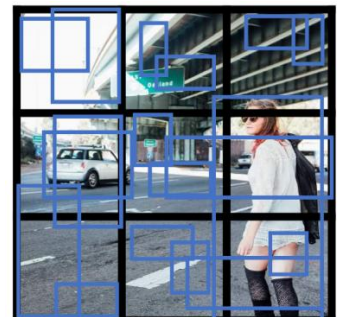
- Training
 - y is grid dimensions $\times$ # anchors $\times$ (5 { p_c , b 's} + # classes)
 - Train a ConvNet that takes a $100 \times 100 \times 3$ image and outputs volume $3 \times 3 \times 16$



- Making predictions



- Outputting the non-max suppressed outputs
 - For each grid cell, get 2 predicted bounding boxes
 - Get rid of low probability predictions
 - For each class (pedestrian, car, motorcycle) use non-max suppression to generate final predictions



Region Proposals

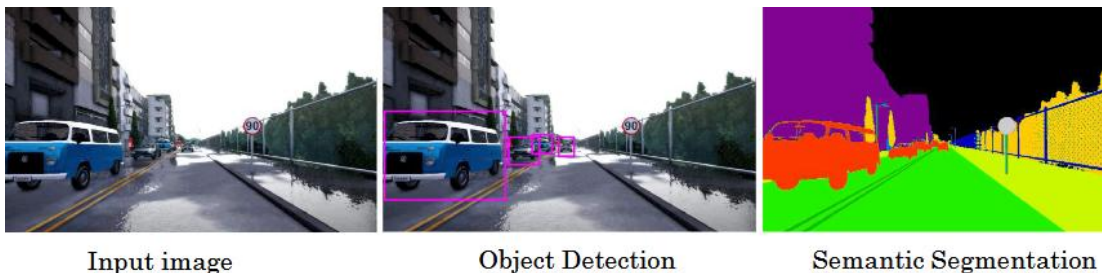
- Used a but less often
- Convolution algorithm classifies a lot of regions where there's clearly no objects
- **Regions with Convolutional Networks (R-CNN)**
 - Tries to pick just a few regions that makes sense to your ConvNet classifier
 - Region proposals via **segmentation algorithm** to figure out what could be objects. Put boxes around blobs and run classifier on just those



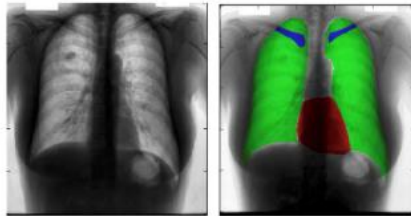
- **R-CNN:** Propose regions, classify proposed regions one at a time, output label + bounding box
- **Fast R-CNN:** Propose regions, use convolution implementation of sliding windows to classify all the proposed regions
- **Faster R-CNN:** use convolutional network to propose regions

Semantic Segmentation with U-Net

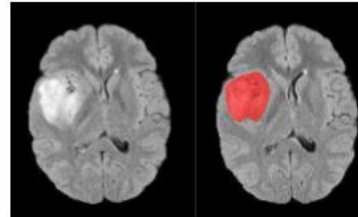
- Draw a careful outline around the object that is detected so that you know exactly which pixel belong to the object
- Object Detection vs. Semantic Segmentation
 - Object detection: boxes around objects
 - Semantic segmentation: label every pixel (e.g. as drivable road, green)



- Motivating example for U-Net: diagnosing disease
 - Determining which pixel correspond to certain parts of the patient's anatomy.
 - Segmentation makes it easier to spot irregularities and diagnose diseases. Also helps with planning surgeries

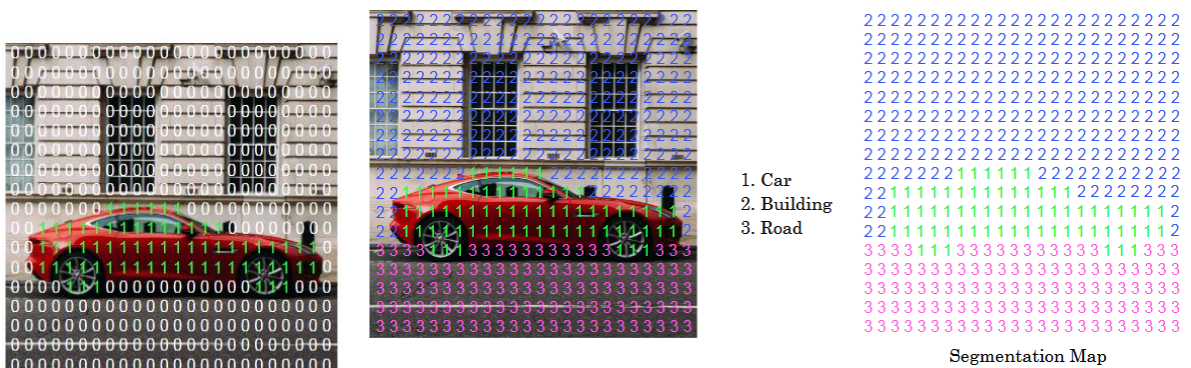


Chest X-Ray

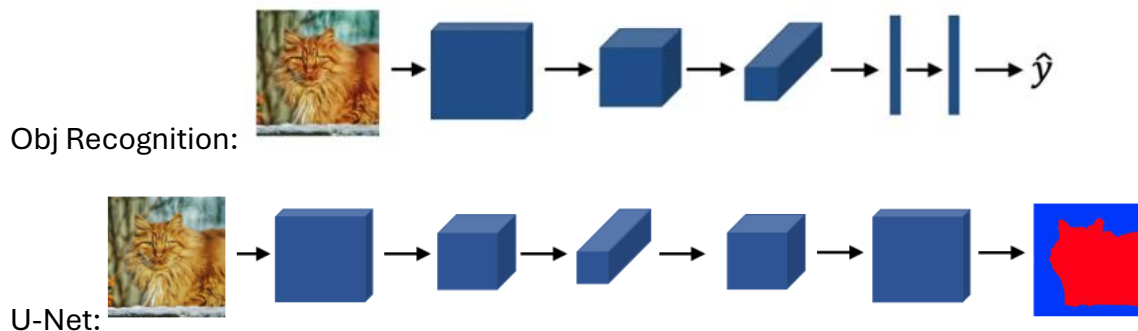


Brain MRI

- Per-pixel class labels
 - Binary classification → U-Net outputs 0 or 1 for every pixel
 - Multiple classification → U-Net matrix of labels 1, 2, ..., c for every pixel



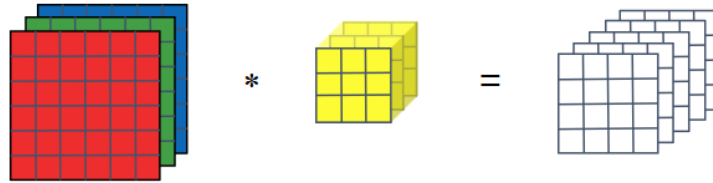
- Deep Learning for Semantic Segmentation
 - Modify object recognition network: remove last few layers
 - Dimensions of image need to get bigger so they can gradually blow back up to full-size image for the output
 - Deeper into U-Net → height/width go up, # channels goes down



Transpose Convolutions

- Key part of U-Net architecture. Blow up 2x2 input into 4x4 output
- Normal convolution shrinks dimensions but has more channels
- Transpose convolution remains flat, but has wider dimensions

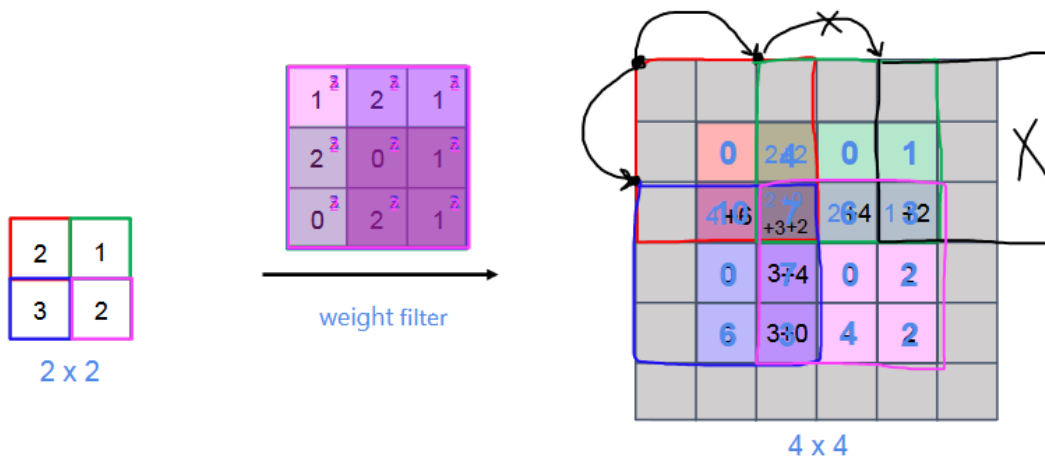
Normal Convolution



Transpose Convolution

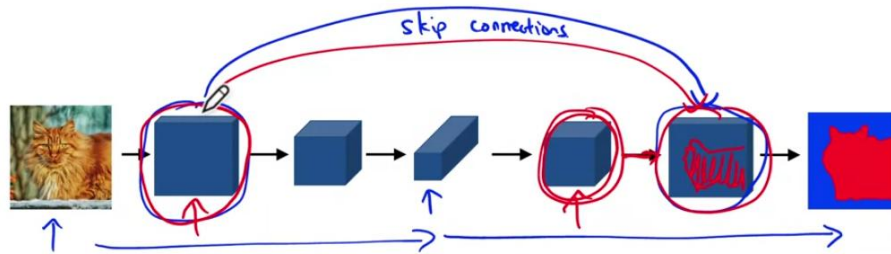


- Instead of placing the filter on the input, you instead place a filter on the output
 - 2 * 3x3 filter → paste 3x3 result in upper left position of output.
 - Padding area won't contain any values, so do not fill in (only use for positioning)
 - 1 * 3x3 filter → paste in output shifted over s=2.
 - When there is some overlap with the previous → add values together
 - Repeat across/down input

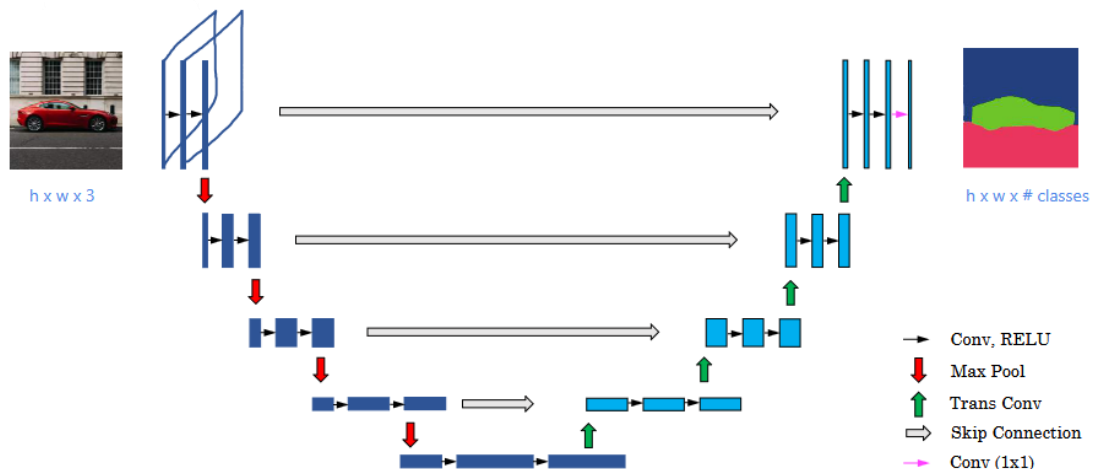


U-Net Architecture

- Intuition: Deep Learning for Semantic Segmentation
 - Normal convolutions for first part of network (compresses image)
 - Second half uses transpose convolution to blow size back up
 - Faster with skip connection. Earlier block of activations is copied directly to later
 - For the final decision layer there are two helpful types of information: high-level spatial/contextual info from previous layer and detailed fine-grain spatial info



- Architecture Diagram
 - Looking at input edge-on (volume behind) for ease of visualization
 - First part of the U-Net uses normal feed forward NN convolutional layers
 - Black arrow = conv layer follow by a ReLU activation function
 - Red arrow = max pooling layer (reduction of height and width)
 - Height and width at bottom of U shape is very small
 - Green arrow = transpose convolutional layer
 - Gray arrow = skip connection (copy activations over to the right)
 - Back to activation set that is the original input image's height and width ($h \times w$)
 - More normal feedforward convolutions, followed by 1×1 conv (pink arrow)
 - Final output is $h \times w \times n_{\text{classes}}$
 - Argmax over n classes to classify each pixel into one of the classes



You are building a 3-class object classification and localization algorithm. The classes are: pedestrian ($c=1$), car ($c=2$), motorcycle ($c=3$). What should y be for the image below? Remember that “?” means “don’t care”, which means that the neural network loss function won’t care what the neural network gives for that component of the output. Recall $y = [p_c, b_x, b_y, b_h, b_w, c_1, c_2, c_3]$.

$$y = [0, ?, ?, ?, ?, ?, ?, ?]$$

$$y = [1, ?, ?, ?, ?, 0, 0, 0]$$

$$y = [1, ?, ?, ?, ?, ?, ?, ?]$$

$$y = [?, ?, ?, ?, ?, ?, ?, ?]$$



You are working on a factory automation task. Your system will see a can of soft-drink coming down a conveyor belt, and you want it to take a picture and decide whether (i) there is a soft-drink can in the image, and if so (ii) its bounding box. Since the soft-drink can is round, the bounding box is always square, and the soft-drink can always appear the same size in the image. There is at most one soft-drink can in each image. Here are some typical images in your training set:



The most adequate output for a network to do the required task is $y = [p_c, b_x, b_y, b_h, b_w, c_1]$. (Which of the following do you agree with the most?)

True, p_c indicates the position of the object and its bounding box, and c_1 indicates the probability of there being a can of soft-drink.

False, since we only need two values c_1 for no soft-drink can and c_2 for soft-drink can.

False, we don't need b_h, b_w since the cans are all the same size.

True, since this is a localization problem.

With the position b_x, b_y we can completely characterize the position of the object if it is present. We should use only one additional logistic unit to indicate if the object is present or not.

When building a neural network that inputs a picture of a person's face and outputs N landmarks on the face (assume that the input image contains exactly one face), we need two coordinates for each landmark, thus we need 2N output units. True/False?

False

True

Each landmark is a specific position in the face's image, thus we need to specify two coordinates for each landmark.

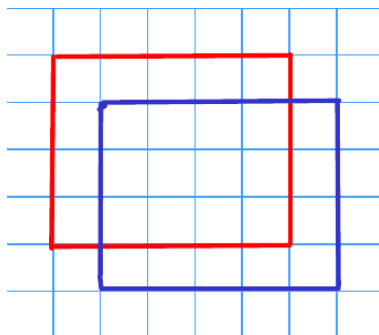
When training one of the object detection systems described in the lectures, you need a training set that contains many pictures of the object(s) you wish to detect. However, bounding boxes do not need to be provided in the training set, since the algorithm can learn to detect the objects by itself.

False

True

You need bounding boxes in the training set. Your loss function should try to match the predictions for the bounding boxes to the true bounding boxes from the training set.

What is the IoU between the red box and the blue box in the following figure? Assume that all the squares have the same measurements.



$$\frac{3}{7}$$

$$\frac{1}{2}$$

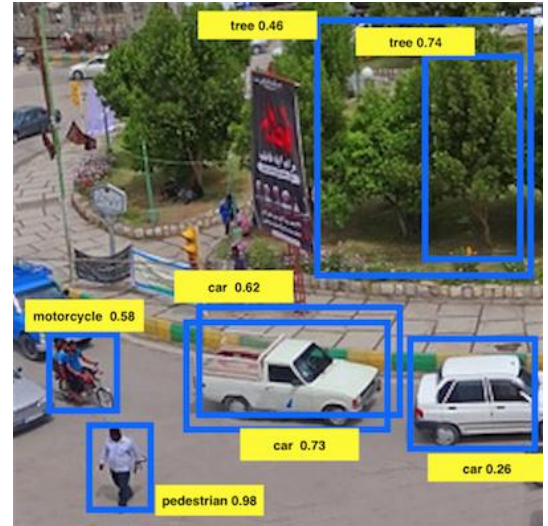
$$\frac{2}{5}$$

$$\frac{4}{5}$$

$$\text{IoU} = \text{intersection} / \text{union} = 12 / (12+16) = 3/7$$

Suppose you run non-max suppression on the predicted boxes below. The parameters you use for non-max suppression are that boxes with probability ≤ 0.4 are discarded, and the IoU threshold for deciding if two boxes overlap is 0.5.

Notice that there are three bounding boxes for cars. After running non-max suppression, only the bounding box of the car with 0.73 is kept from the three bounding boxes for cars. True/False?



True. The non-maximum suppression eliminates the bounding boxes with scores lower than the ones of the maximum.

False. All the cars are eliminated since there is a pedestrian with a higher score of 0.98.

False. Two bounding boxes corresponding to cars are left since their IoU is zero.

One of the bounding boxes for cars is eliminated because it has a lower score and an IoU higher than 0.5.

If we use anchor boxes in YOLO we no longer need the coordinates of the bounding box b_x, b_y, b_h, b_w since they are given by the cell position of the grid and the anchor box selection. True/False?

False True

We use the grid and anchor boxes to improve the capabilities of the algorithm to localize and detect objects, for example, two different objects that intersect, but we still use the bounding box coordinates.

Semantic segmentation can only be applied to classify pixels of images in a binary way as 1 or 0, according to whether they belong to a certain class or not. True/False?

True **False**

The ideas for multi-class classification can be applied to semantic segmentation.

Using the concept of Transpose Convolution, fill in the values of **X**, **Y** and **Z** below.

(padding = 1, stride = 2)

Input: 2x2

1	2
3	4

Filter: 3x3

1	1	1
0	0	0
-1	-1	-1

Result: 6x6

	0	0	0	X	
	Y	4	2	2	
	0	0	0	0	
	-3	Z	-4	-4	

X = 0, Y = 2, Z = -1

X = 0, Y = -1, Z = -4

X = 0, Y = 2, Z = -7 $Y = 2 \times 1$ $Z = 3(-1) + 4(-1) = -7$

X = 0, Y = -1, Z = -7

When using the U-Net architecture with an input $h \times w \times c$, where c denotes the number of channels, the output will always have the shape $h \times w \times c$. True/False?

False True

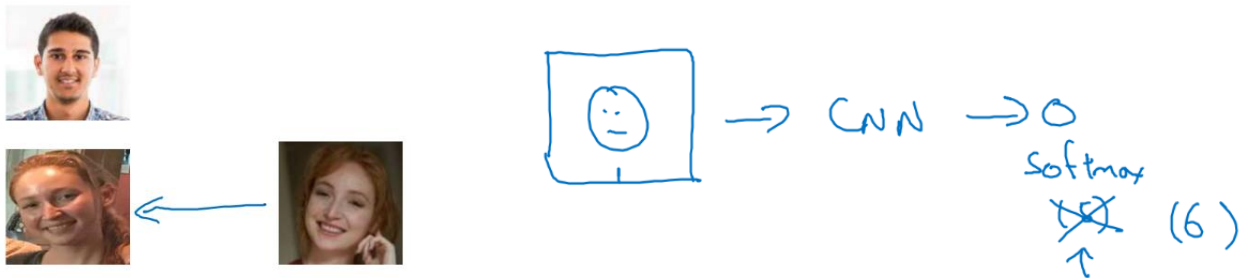
The output of the U-Net architecture can be $h \times w \times k$ where k is the number of classes. The number of channels doesn't have to match between input and output.

Face Recognition

- **Verification:** Input image and name/ID; output whether the input image is of the claimed person (1:1 problem)
- **Recognition:** Has a database of K persons, get an input image, output ID if the image is any of the K persons (or “not recognized”) (1:K problem)
- Verification is a building block towards recognition

One-Shot Learning

- Learning from one example to recognize the person again
 - Softmax output for each of the people and none of the above doesn't work well. Not enough data, and if a new person joins you have to retrain ConvNet

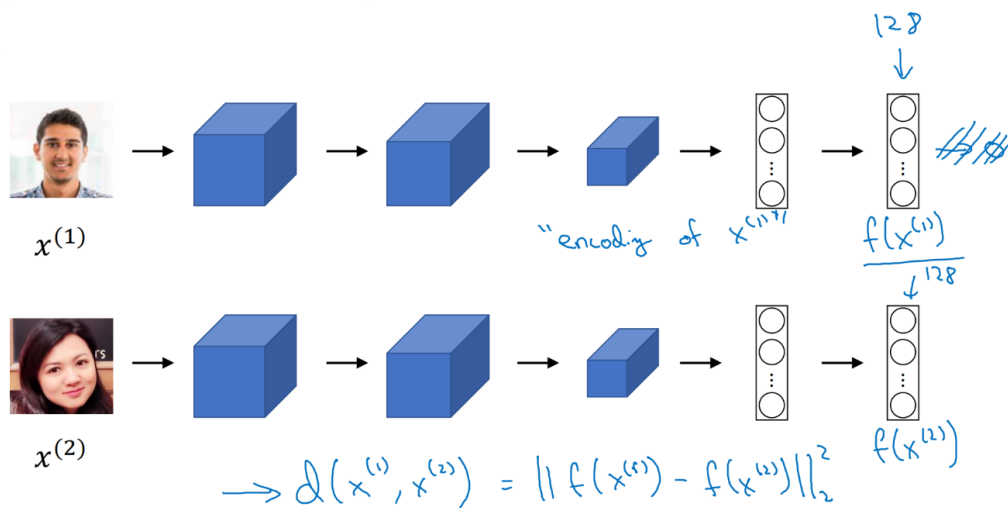


- Learning a “**similarity**” function
 - $d(\text{img1}, \text{img2}) = \text{degree of difference between images}$
 - $d(\text{img1}, \text{img2}) \leq \tau \rightarrow \text{same person}$
 - $d(\text{img1}, \text{img2}) > \tau \rightarrow \text{different people}$
 - Match: d small for one comparison
 - No matches: d large for all comparisons
 - If you can learn a function d that inputs a pair of images and outputs whether they're the same person, then you can add more people without retraining



Siamese Network

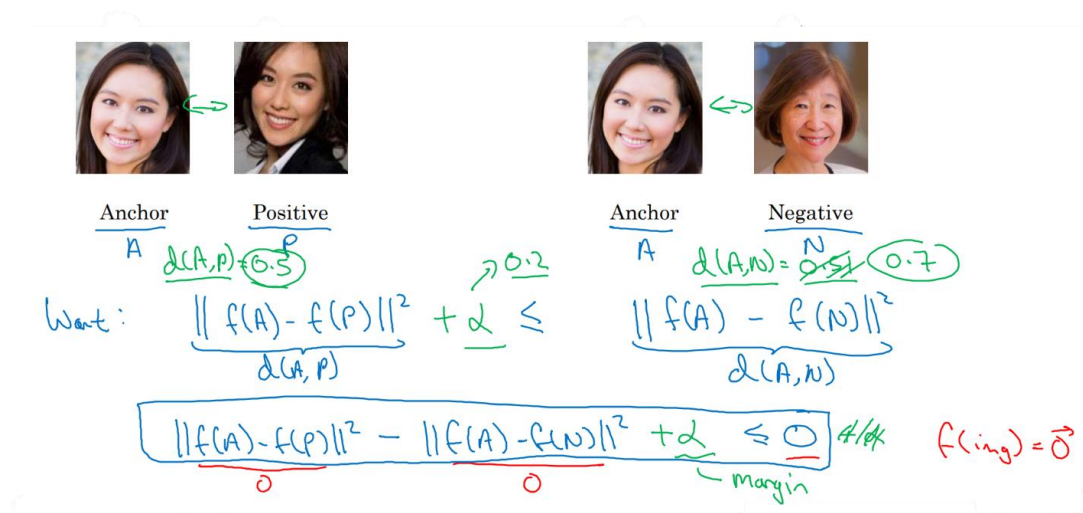
- Running two identical CNNs on two different input images and comparing them
- Ex: tell how similar to different two faces are (DeepFace)
 - $f(x^{(1)})$ = encoding of input image $x^{(1)}$ (here a vector, not softmax output)
 - Feed 2nd picture to same NN (same parameters) and get a different vector $f(x^{(2)})$
 - Distance between x_1 and x_2 is the norm of the difference between the encodings
 - $d(x^{(1)} - x^{(2)}) = ||f(x^{(1)}) - f(x^{(2)})||^2$
 - Train NN so that the encoding it computes results in a function d that tells you when two pictures are of the same person



- Goal of learning
 - Parameters of NN define an encoding $f(x^{(i)})$
 - Learn parameters so that:
 - If $x^{(i)}, x^{(j)}$ are the same person, F is small
 - If $x^{(i)}, x^{(j)}$ are different people, $||f(x^{(i)}) - f(x^{(j)})||^2$ is large
 - As you vary the parameters in the layers of the NN, you end up with different encodings. Use backprop to vary these parameters to make sure the conditions are satisfied

Triplet Loss

- How do you define an objective function to make a NN learn this?
- One way to learn the params of the NN so that it gives out a good encoding is to define and apply gradient descent on the triplet loss function
- Learning Objective
 - Compare pairs of images. Want encodings to be similar to the same person and different for different people
 - Always look at one anchor image and measure distance. Distance between anchor and positive example (same person) small, distance between anchor and negative example (different person) large
 - Triplet: Anchor (A), Positive (P), Negative (N)
 - Want: $\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2$ $[d(A,P) + \alpha \leq d(A,N)]$
 $\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha \leq 0$
 - Adding margin α prevents it from learning trivial solutions (set everything to 0 or the same for every image). Pushes A,P and A,N pairs away from each other



- Loss function:** given 3 images A, P, N
 $L(A, P, N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$ [either 0 or ≤ 0]

- Cost function:** $J = \sum L(A^{(i)}, P^{(i)}, N^{(i)})$
 - Training set: 10k pictures of 1k persons (10 pictures of each), then apply with one-shot

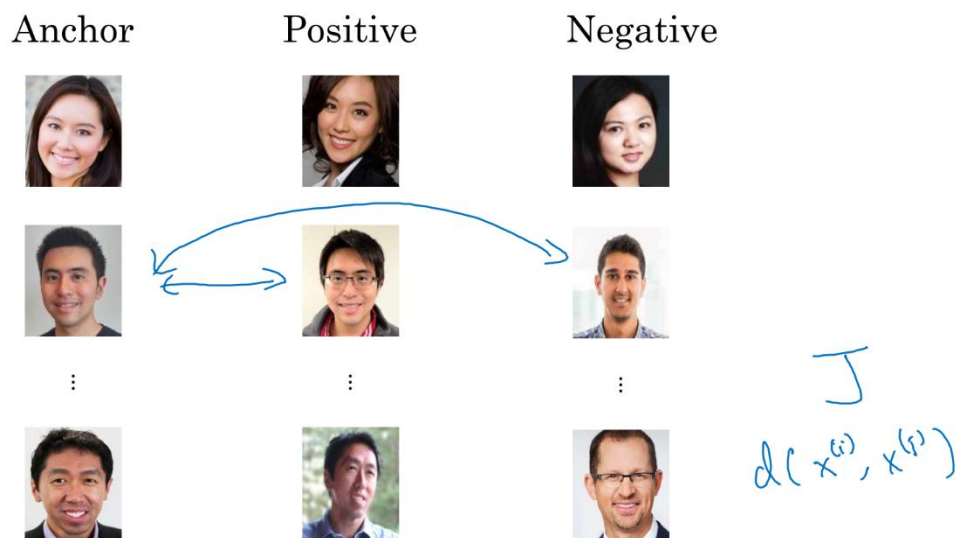
$$L(A, P, N) = \max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$$

$$J = \sum_{i=1}^m L(A^{(i)}, P^{(i)}, N^{(i)})$$

Training set: 10k pictures of 1k persons

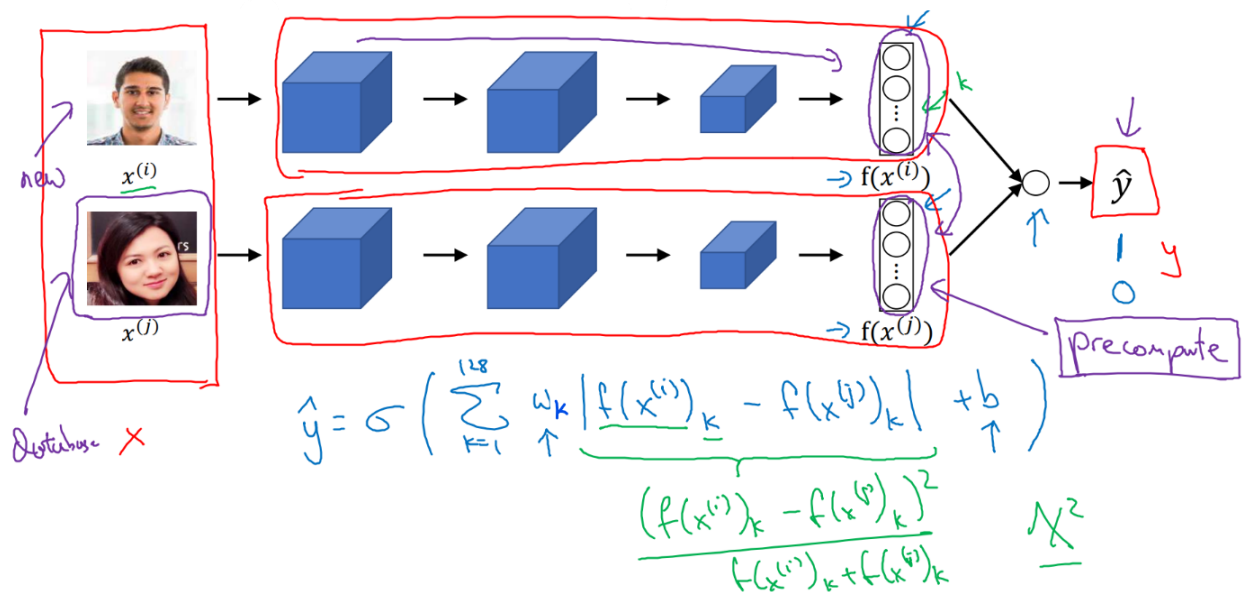
Handwritten notes show arrows pointing from the training set description to the variables $A^{(i)}, P^{(i)}, N^{(i)}$ in the cost function equation.

- Choosing the triplets A, P, N
 - During training, if A,P,N are chosen randomly, $d(A, P) + \alpha \leq d(A, N)$ is easily satisfied [$\|f(A) - f(P)\|^2 + \alpha \leq \|f(A) - f(N)\|^2$]
 - Choose triplets that are “hard” to train on [$d(A, P) + \alpha \approx d(A, N)$]. Gradient descent has to do some work
 - Increases computational efficiency
- Training set using triplet loss
 - GD to minimize cost function → backpropagating to all NN parameters to learn an encoding such that d of two images will be small when they’re of the same person and large when images of different people



Face Verification and Binary Classification

- Triplet loss is one good way to learn the parameters of a ConvNet for face recognition, but it can also be posed as a straight binary classification problem
- Learning the similarity function
 - Have pair of NNs compute embeddings and have these be the input to a logistic regression unit to make prediction (where the target output is 1 if same person, 0 if different people). This final logistic regression unit outputs a sigmoid function applied to differences in encodings (element-wise abs value or chi-squared similarity) treated as features
 - Input is training image pair → Siamese network → output is 0 or 1
 - Precompute embedding so that when new employee walks in, other ConvNet computes encoding and compares to pre-computed to make prediction $\hat{y}$

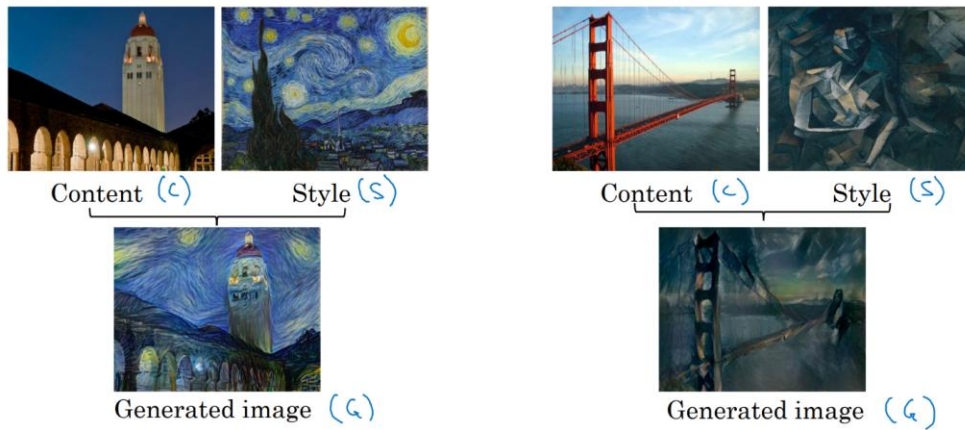


- Face verification supervised learning
 - Create training set of pairs of images, where target label is 1 when same and 0 when different
 - Use different pairs to train Siamese network using backpropagation

x		y	
		1	"Same"
		0	"Different"
		0	
		1	

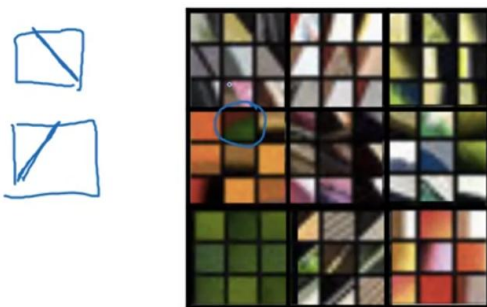
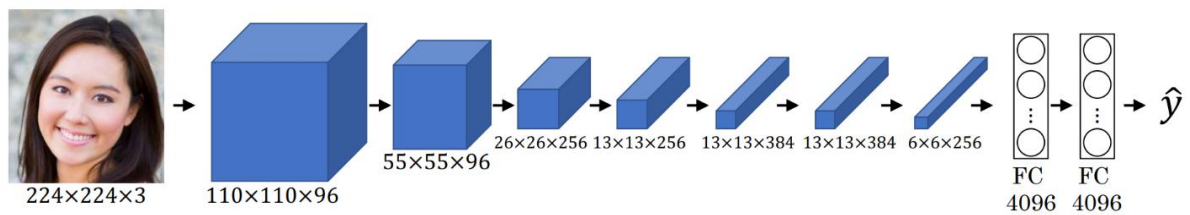
Neural Style Transfer

- Generate new image (G) of content (C) in the a style (S)
- Need to look at the features extracted by the ConvNet at various layers



What Are Deep ConvNets Learning?

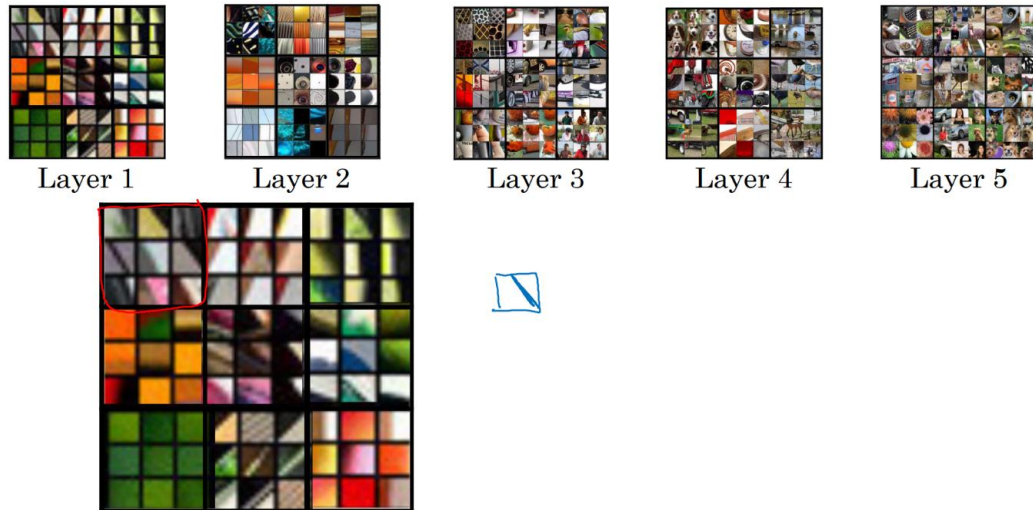
- Visualizing what a deep network is learning
 - Pick a unit in layer 1, find the nine image patches that maximize the unit's activation; repeat for other units
 - Looks for simple features like edges or color
 - In deeper layers, a hidden unit will see a larger region of the image



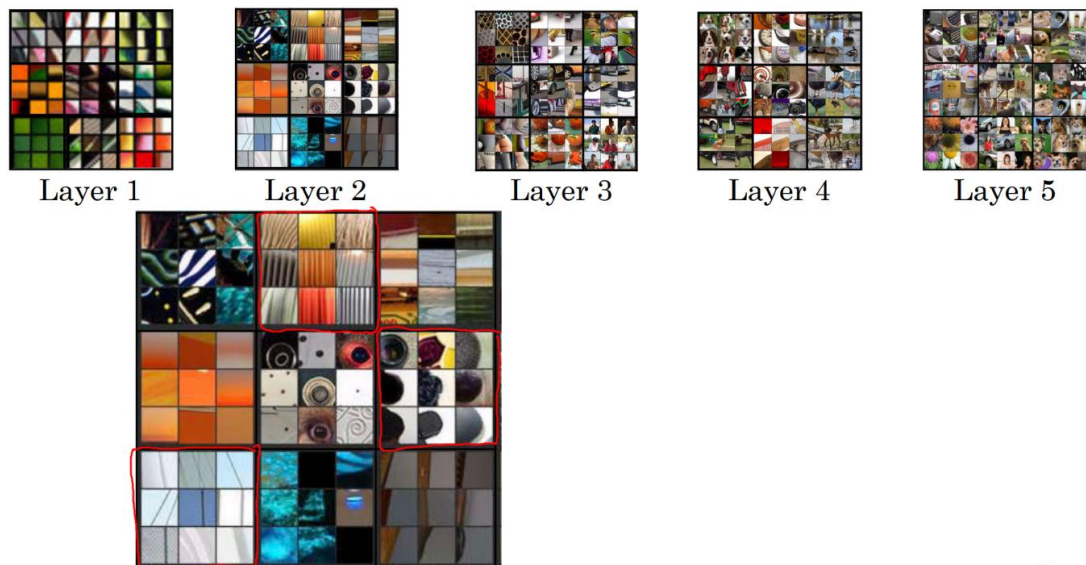
- Visualizing deep layers

- 9 hidden patches that cause one hidden unit to be highly activated. Each grouping of 9 is a different set of nine image patches that cause one hidden unit to be activated

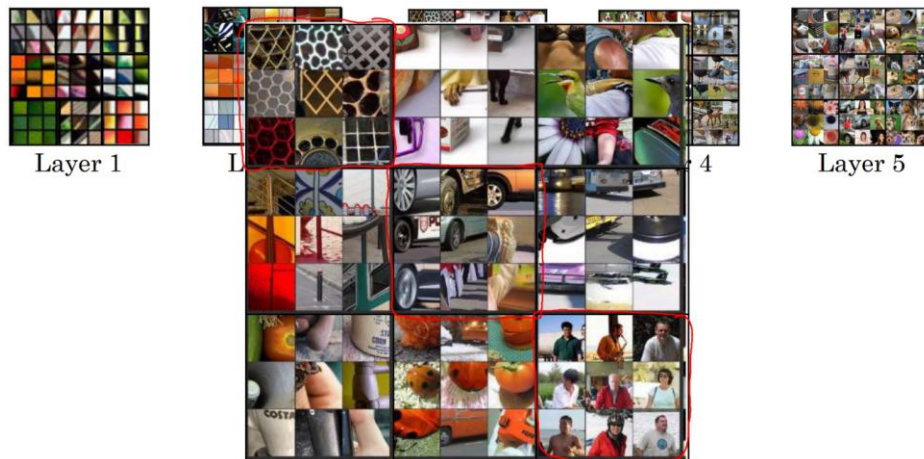
- Layer 1: edges/colors



- Layer 2: basic shapes (vertical texture, round shape on left, thin vertical line)



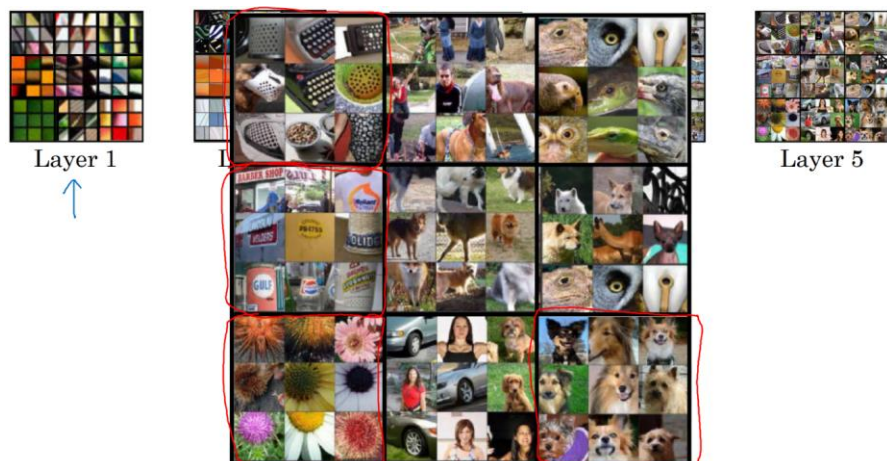
- Layer 3: more complex features (round in lower left, people, honeycomb shapes)



- Layer 4: even more complex (dogs, spirals, water, bird legs)



- Layer 5: sophisticated things (more varied dogs, keyboards/dots, text, flowers)



Neural Style Transfer Cost Function

- Generated Image Cost Function

- $J(G) = \alpha J_{\text{content}}(\mathbf{C}, \mathbf{G}) + \beta J_{\text{style}}(\mathbf{S}, \mathbf{G})$
- $J_{\text{content}}(\mathbf{C}, \mathbf{G})$ measures how similar the contents of the generated image are to the content image $\mathbf{C}$
- $J_{\text{style}}(\mathbf{S}, \mathbf{G})$ measures how similar the style of the generated image is to the style of image $\mathbf{S}$
- α and β are hyperparameters that specify relative weighting between the content cost and style cost



Content $\mathbf{C}$



Style $\mathbf{S}$



Generated image $\mathbf{G}$

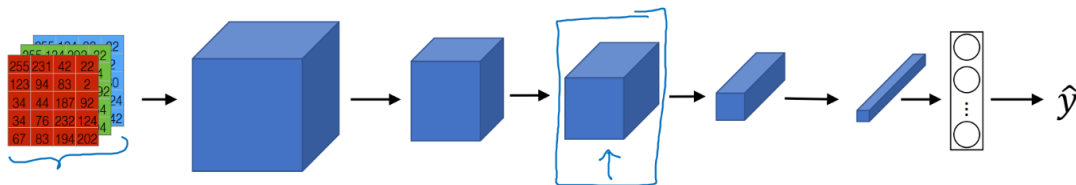
- Find the generated image $\mathbf{G}$
 - Initiate $\mathbf{G}$ randomly (e.g. $\mathbf{G}$: $100 \times 100 \times 3$)
 - Use gradient descent to minimize $J(\mathbf{G})$
 - $\mathbf{G} := \mathbf{G} - \partial/\partial \mathbf{G} J(\mathbf{G})$

- Content Cost Function

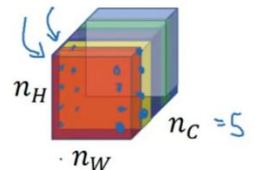
- $J(\mathbf{G}) = \alpha J_{\text{content}}(\mathbf{C}, \mathbf{G}) + \beta J_{\text{style}}(\mathbf{S}, \mathbf{G})$
- Say you use hidden layer l (typically middle) to compute content cost
- Use pre-trained ConvNet (e.g., VGG network)
- Let $\mathbf{a}^{[l](\mathbf{C})}$ and $\mathbf{a}^{[l](\mathbf{G})}$ be the activation of layer l on the images
- If $\mathbf{a}^{[l](\mathbf{C})}$ and $\mathbf{a}^{[l](\mathbf{G})}$ are similar, both images have similar content
- $J_{\text{content}}(\mathbf{C}, \mathbf{G}) = \frac{1}{2} \|\mathbf{a}^{[l](\mathbf{C})} - \mathbf{a}^{[l](\mathbf{G})}\|^2$ (element-wise sum of squares of differences, normalization constant optional)
- Incentivizes algorithm to find an image $\mathbf{G}$ such that the hidden layer activations are similar to what to got for the content image

- Style Cost Function

- Meaning of the “style” of an image



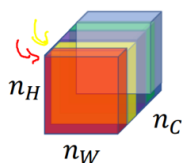
- Say you are using layer l 's activation to measure “style.” Define **style** as correlation between activations across channels
- How correlated are the activations across different channels?



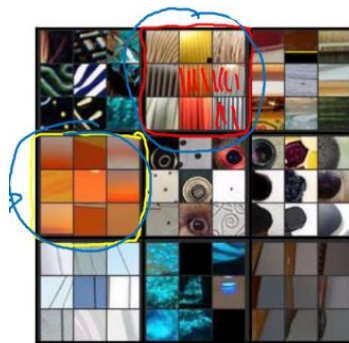
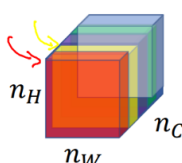
- Intuition about style of an image

- Let's say red channel corresponds to the 2nd neuron detecting vertical texture and the yellow channel corresponds to the 4th neuron detecting orange patches
- Correlated: whatever part of the image has the subtle vertical texture, that part of the image will probably have an orange tint
- Uncorrelated: whenever there is vertical texture, probably not orange
- Correlations tells which high level texture components tend to occur or not occur together in part of an image
- Degree of correlation between channels is a measure of style
- Measure the degree to which in your generated image, the first channel is correlated or uncorrelated with the second channel. This tells you in the generated image how often the vert texture occurs/doesn't with orange

Style image



Generated Image



○ Style matrix

- Let $a_{l,j,k}^{[l]}$ = activation at (l, j, k) [H, W, C indices]. $G^{[l]}$ is $n_c^{[l]} \times n_c^{[l]}$
- $G_{kk'}^{[l]}$ measures how correlated the activations in channel k are compared to the activations in channel k' , where k and k' range from 1 to the number of channels in that layer
- $G_{kk'}^{[l]}$ sums over the different positions of the image (H and W) and multiplying the activations of channels k and k' together (unnormalized cross-covariance)
 - If the activations are large together, G will be large
 - If uncorrelated, G might be small
 - Calculate for style image and generated image
 - Resulting style matrices are called gram matrices in linear algebra
- Style cost function is the sum of squares of the element-wise difference between the two style matrices (with normalization constant $2 n_H^{[l]} n_W^{[l]} n_C^{[l]}$ that doesn't matter much since the cost is multiplied by hyperparameter β anyway)

n_c
 $G_{kk'}^{[l]}$
 $k = 1, \dots, n_c$

$$\begin{aligned} G_{kk'}^{[l]}(s) &= \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{ijk}^{[l]}(s) a_{ijk'}^{[l]}(s) \\ G_{kk'}^{[l]}(G) &= \sum_{i=1}^{n_H^{[l]}} \sum_{j=1}^{n_W^{[l]}} a_{ijk}^{[l]}(G) a_{ijk'}^{[l]}(G) \end{aligned}$$

"Gram matrix"

$$\begin{aligned} J_{\text{style}}^{[l]}(s, G) &= \frac{1}{2} \| G_{kk'}^{[l]}(s) - G_{kk'}^{[l]}(G) \|_F^2 \\ &= \frac{1}{(2 n_H^{[l]} n_W^{[l]} n_C^{[l]})^2} \sum_k \sum_{k'} (G_{kk'}^{[l]}(s) - G_{kk'}^{[l]}(G))^2 \end{aligned}$$

β ↑

- Style cost function

- Frobenius norm
- Weight by additional parameters $\lambda^{[l]}$
- Use multiple layers from different parts of NN to compute style

$$\|G^{[l](S)} - G^{[l](G)}\|_F^2$$

$$J_{style}^{[l]}(S, G) = \frac{1}{(2n_H^{[l]}n_W^{[l]}n_C^{[l]})^2} \sum_k \sum_{k'} (G_{kk'}^{[l](S)} - G_{kk'}^{[l](G)})^2$$

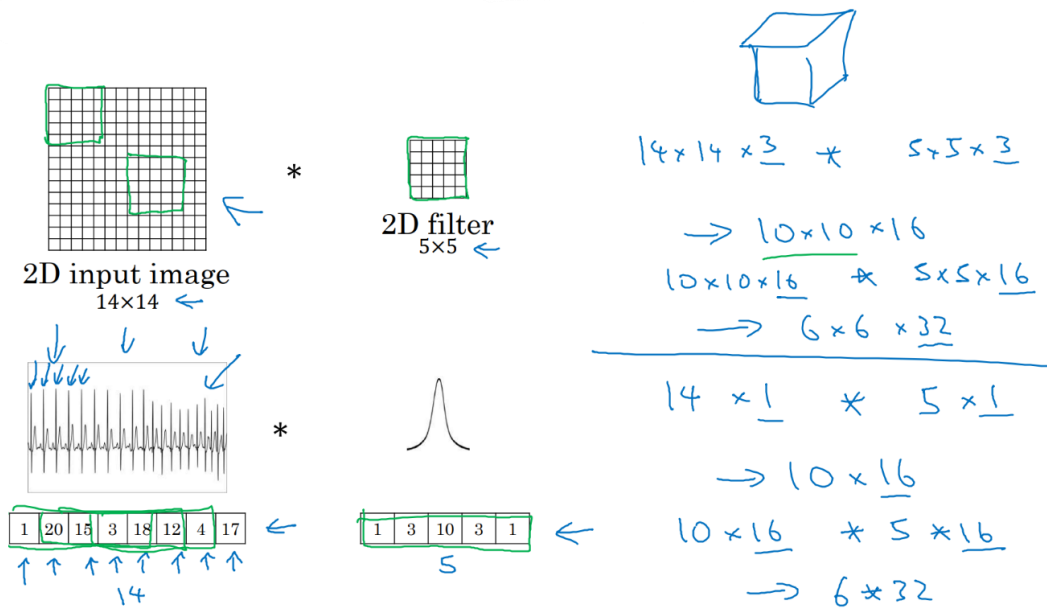
$$J_{style}(S, G) = \sum_l \lambda^{[l]} J_{style}^{[l]}(S, G)$$

$$\underbrace{J(G)}_G = \alpha J_{content}(G, G) + \beta J_{style}(S, G)$$

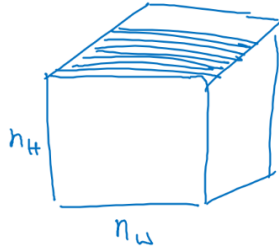
1D and 3D Generalization

- Convolutions in 2D and 1D

- EKG data is 1D voltage data. Want a 1D filter
- Similarly apply 1D filter across lot of different positions throughout the 1D signal
- Similar dimensions to 2D example (1D: $14 * 5 \rightarrow 10$; 2D: $14 \times 14 * 5 \times 5 \rightarrow 10 \times 10$)
- Usually use RNNs for 1D, but can use CNNs as well



- 3D convolution
 - 3D data is made up of slices



- Apply a 3D filter that detects features across 3D data
- Also applies to video data, where the volume dimension is time
- $14 \times 14 \times 14 (x1) * 5 \times 5 \times 5 (x1) \rightarrow 10 \times 10 \times 10 (x16)$



3D volume



*



3D filter

$$\begin{array}{l}
 \downarrow \quad \downarrow \quad \downarrow \quad \downarrow n_c \\
 \underline{14 \times 14 \times 14} \times 1 \\
 * \underline{5 \times 5 \times 5} \times 1 \quad 16 \text{ filters} \\
 \rightarrow 10 \times 10 \times 10 \times \underline{16} \\
 * 5 \times 5 \times 5 \times \underline{16} \quad 32 \text{ filters} \\
 \rightarrow 6 \times 6 \times 6 \times 32
 \end{array}$$

Face verification and face recognition are the two most common names given to the task of comparing a new picture against one person's face. True/False?

False True

This is the description of face verification, but not of face recognition.

Why do we learn a function $d(img1, img2)$ for face verification? (Select all that apply.)

Given how few images we have per person, we need to apply transfer learning.

This allows us to learn to predict a person's identity using a softmax output unit, where the number of classes equals the number of persons in the database plus 1 (for the final "not in database" class).

We need to solve a one-shot learning problem.

This allows us to learn to recognize a new person given just a single image of that person.

You want to build a system that receives a person's face picture and determines if the person is inside a workgroup. You have pictures of all the faces of the people currently in the workgroup, but some members might leave, and some new members might be added. To train a system to solve this problem using the triplet loss you get many persons and take several pictures of each one. Which of the following do you agree with? (Select the best answer.)

You take several pictures of the same person because this way you can get more pictures to train the network efficiently since you already have the person in place.

You shouldn't use persons outside the workgroup you are interested in because that might create a high variance in your model.

It would be best to increase the number of persons in the dataset by taking only one picture of each person to have a more representative set of the population.

You take several pictures of the same person to train $d(img_1, img_2)$ using the triplet loss.

In the triplet loss: $\max(\|f(A) - f(P)\|^2 - \|f(A) - f(N)\|^2 + \alpha, 0)$. Which of the following are true about the triplet loss? Choose all that apply.

We want that $\|f(A) - f(P)\|^2 < \|f(A) - f(N)\|^2$ so the negative images are further away from the anchor than the positive images.

Being a positive image, the encoding of P should be close to the encoding of A.

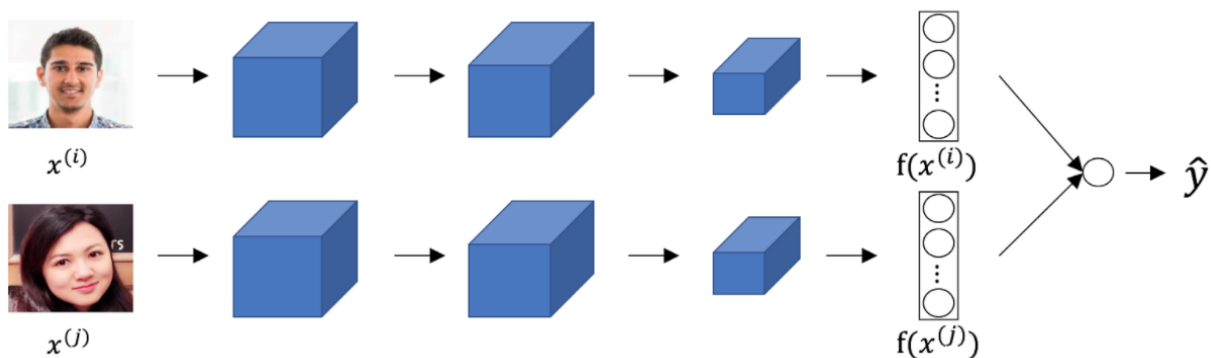
A, the anchor image, is a hyperparameter of the Siamese network.

α is a trainable parameter of the Siamese network.

$f(A)$ represents the encoding of the Anchor.

f represents the network that is in charge of creating the encoding of the images, and A represents the anchor image.

Consider the following Siamese network architecture:



The upper and lower neural networks have different input images, but have exactly the same parameters.

True False

Parameters are shared

You train a ConvNet on a dataset with 100 different classes. You wonder if you can find a hidden unit which responds strongly to pictures of cats. (i.e., a neuron so that, of all the input/training images that strongly activate that neuron, the majority are cat pictures.) You are more likely to find this unit in layer 4 of the network than in layer 1.

True **False**

Neurons that understand more complex shapes are in deeper layers

Neural style transfer is trained as a supervised learning task in which the goal is to input two images (x), and train a network to output a new, synthesized image (y).

True **False**

Neural style transfer is about training the pixels of an image to make it look artistic, it is not learning any parameters.

In neural style transfer, we define style as:

$\| a^{[l](S)} - a^{[l](G)} \|^2$ the distance between the activation of the style image and the content image.

The correlation between the activation of the content image C and the style image S.

The correlation between activations across channels of an image.

correlation: $G_{kk}^{[l](I)}$ for image I

The correlation between the generated image G and the style image S.

In neural style transfer, what is updated in each iteration of the optimization algorithm?

The pixel values of the content image C

The pixel values of the generated image G

The neural network parameters

The regularization parameters

Neural style transfer doesn't learn any parameters. It learns the pixels of an image

You are working with 3D data. The input "image" has size $64 \times 64 \times 64 \times 3$, if you apply a convolutional layer with 16 filters of size $4 \times 4 \times 4$, zero padding and stride 2. What is the size of the output volume?

$$61 \times 61 \times 61 \times 14$$

$$64 \times 64 \times 64 \times 3$$

$$31 \times 31 \times 31 \times 3$$

$$\mathbf{31 \times 31 \times 31 \times 16}$$

$$n^{[l]} = [(n^{[l-1]} - f + 2p) / s] + 1 = [(64 - 4 + 2*0) / 2] + 1 = 31 \quad \text{Filters} = 16$$